DebCovDiff: Differential Testing of Coverage Measurement Tools on Real-World Projects

Wentao Zhang, Jinghao Jia, Erkai Yu, Darko Marinov, Tianyin Xu

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

NSF
CNS-1956007
CCF-1956374
CNS-2145295

ASE '25 (to appear)

BOEING
Grant 2023-BRT-PA-064

Code Coverage 101

Function under test:

Test cases:

Code Coverage 101

Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

Code Coverage 101

Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

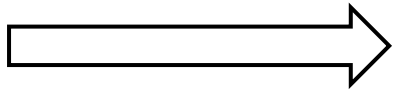
```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Code Coverage 101

Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Coverage tools



Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Code Coverage 101

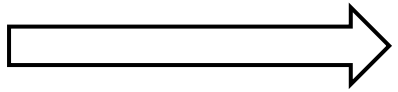
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



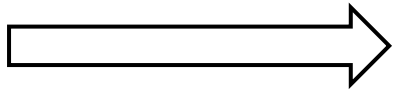
```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Code Coverage 101

Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
1 |         return 3;
2 |     return 4;
3 | }
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Code Coverage 101

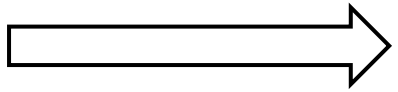
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
1 |         return 3;
2 |     return 4;
3 | }
```

Line coverage
How many times is each line executed?

Code Coverage 101

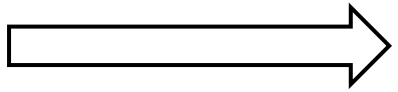
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
1 |         return 3;
2 |     return 4;
3 | }
```

Line coverage
How many times is each line executed?

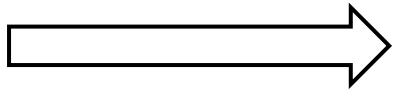
Fully covered

Code Coverage 101

Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Coverage tools



Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Code Coverage 101

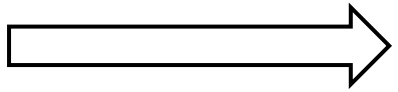
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
1 |         return 3;
2 |     return 4;
3 | }
```

Code Coverage 101

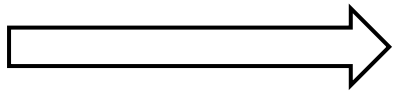
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
    [True: 2, False: 1]
    [True: 0, False: 2]
    [True: 1, False: 2]
1 |         return 3;
2 |     return 4;
3 | }
```

Code Coverage 101

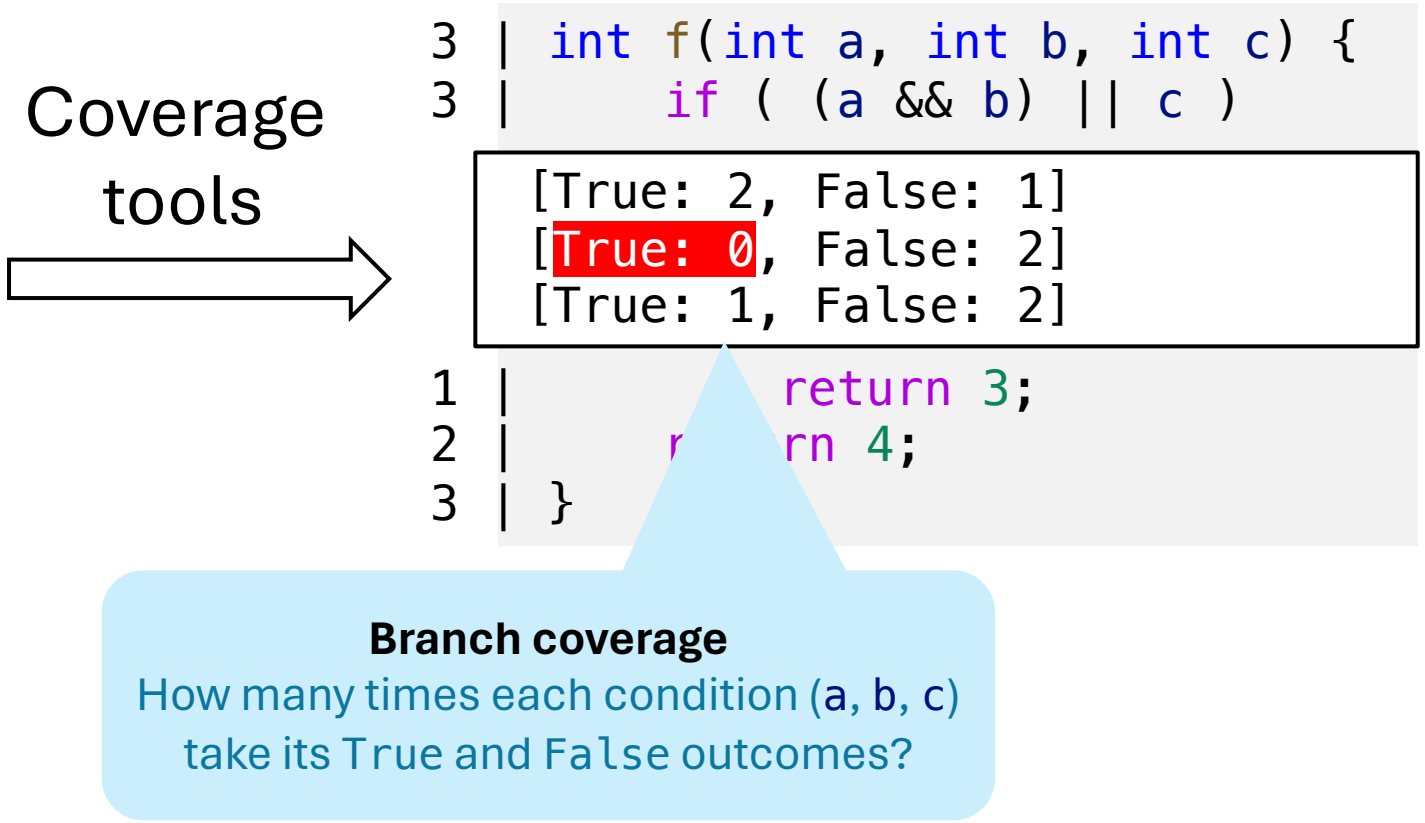
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



Code Coverage 101

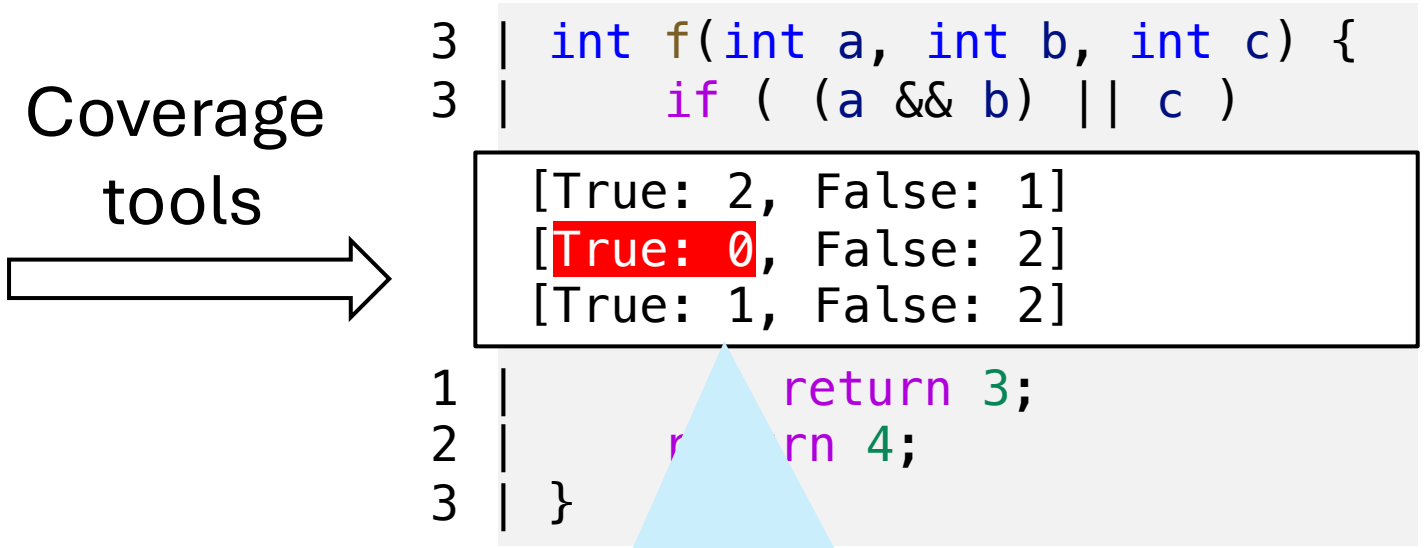
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



Branch coverage

How many times each condition (a, b, c) take its True and False outcomes?

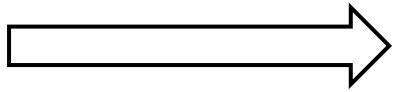
Some missed

Code Coverage 101

Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Coverage tools



Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Code Coverage 101

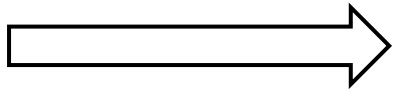
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
1 |         return 3;
2 |     return 4;
3 | }
```

Code Coverage 101

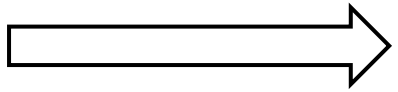
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )

     C1, C2, C3   Result
 1 { F , - , F  = F     }
 2 { T , F , F  = F     }
 3 { T , F , T  = T     }

 C1-Pair: not covered
 C2-Pair: not covered
 C3-Pair: covered: (2,3)

1 |         return 3;
2 |     return 4;
3 | }
```

Code Coverage 101

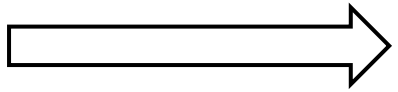
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
```

```
     C1, C2, C3   Result
1 { F , - , F  = F     }
2 { T , F , F  = F     }
3 { T , F , T  = T     }

C1-Pair: not covered
C2-Pair: not covered
C3-Pair: covered: (2,3)
```

```
1 |         return 3;
2 |     return 4;
3 | }
```

MC/DC

Does each condition take both outcomes, while the rest of conditions stay the same, and the result differs?

Code Coverage 101

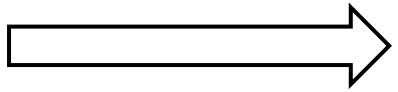
Function under test:

```
int f(int a, int b, int c) {
    if ( (a && b) || c )
        return 3;
    return 4;
}
```

Test cases:

```
assert( f(0, 0, 0) == 4 );
assert( f(1, 0, 0) == 4 );
assert( f(1, 0, 1) == 3 );
```

Coverage tools



```
3 | int f(int a, int b, int c) {
3 |     if ( (a && b) || c )
```

```
     C1, C2, C3   Result
1 { F , - , F  = F     }
2 { T , F , F  = F     }
3 { T , F , T  = T     }

C1-Pair: not covered
C2-Pair: not covered
C3-Pair: covered: (2,3)
```

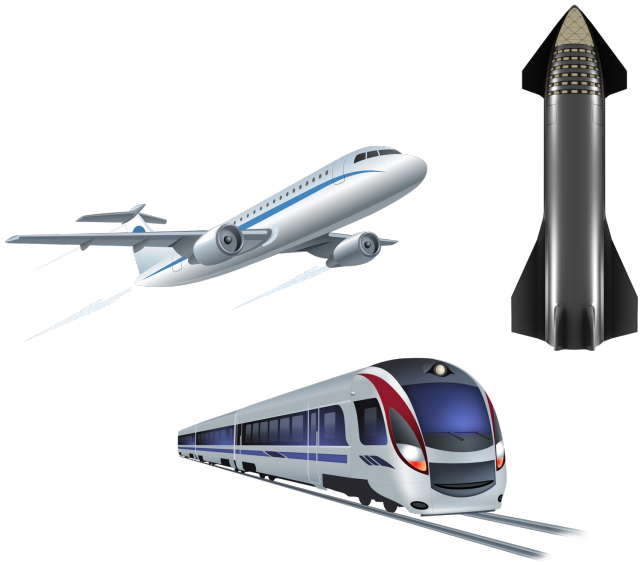
```
1 |         return 3;
2 |     return 4;
3 | }
```

MC/DC

Does each condition take both outcomes, while the rest of conditions stay the same, and the result differs?

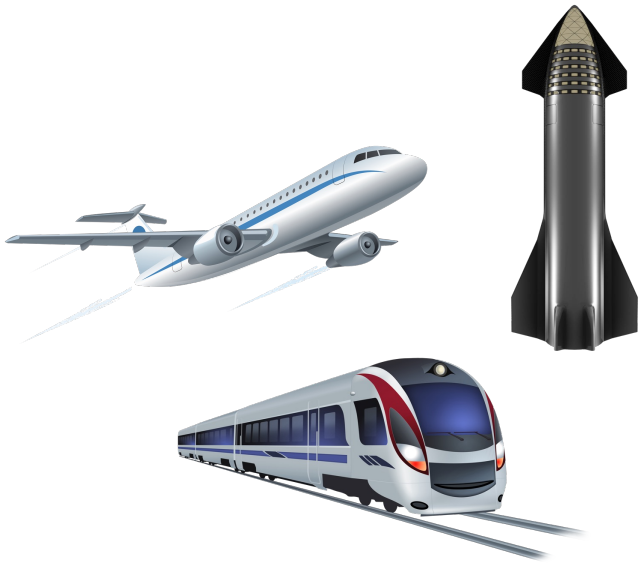
More missed

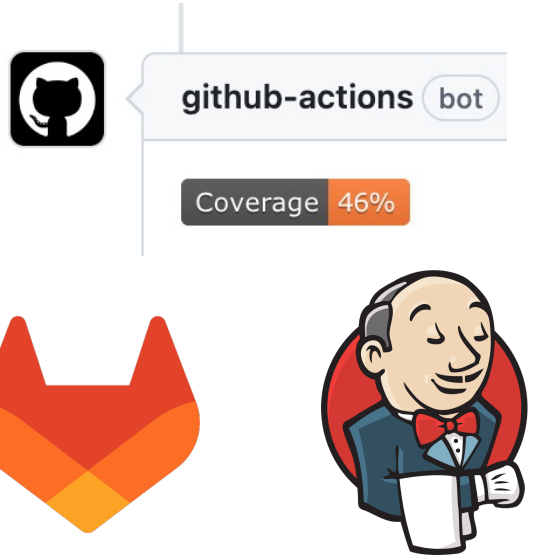
Why Code Coverage Matters



- **Certification** in safety-critical industries
  - Aviation (DO-178C)
  - Automotive (ISO-26262)

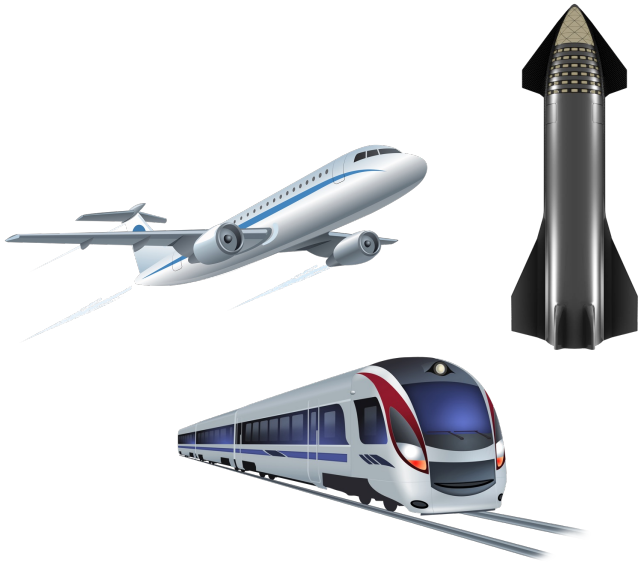
Why Code Coverage Matters



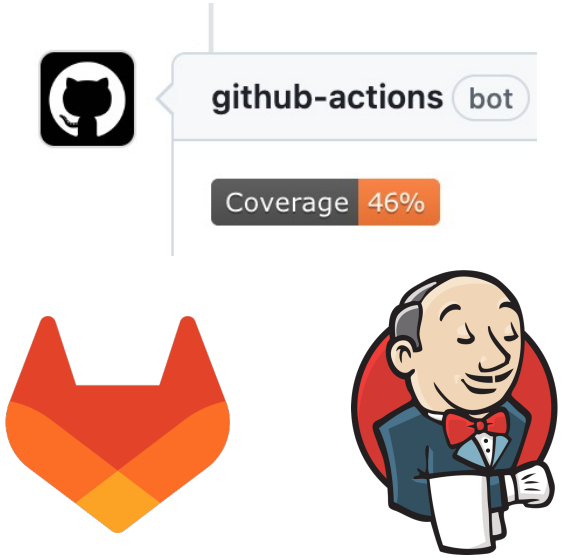


- **Certification** in safety-critical industries
  - Aviation (DO-178C)
  - Automotive (ISO-26262)
- Everyday **development**
  - Continuous integration (GitHub Actions)
  - Tech companies such as Google [Ivanković et al. 2019]

Why Code Coverage Matters



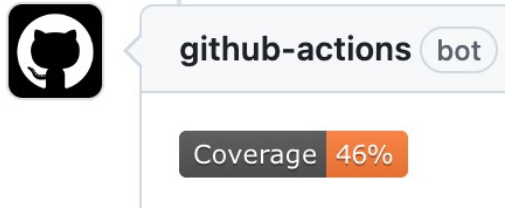
- **Certification** in safety-critical industries
  - Aviation (DO-178C)
  - Automotive (ISO-26262)

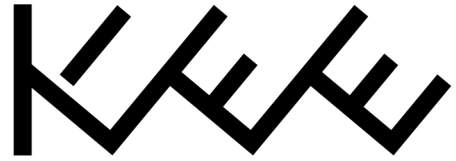


- Everyday **development**
  - Continuous integration (GitHub Actions)
  - Tech companies such as Google [Ivanković et al. 2019]

- Advanced **program analysis**
  - Fuzzing (AFL, `libFuzzer`)
  - Fault localization [Jones et al. 2005]

Why Code Coverage Matters





...only when the reported coverage is *correct*!

critical industries

- Aviation (DO-178C)
- Automotive (ISO-26262)

- Continuous integration (GitHub Actions)
- Tech companies such as Google [Ivanković et al. 2019]

analysis

- Fuzzing (AFL, `libFuzzer`)
- Fault localization [Jones et al. 2005]

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Snippet under test:

“Test case”:

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Snippet under test:

```
/* Skip IPv6 link-local addresses */
if (family == AF_INET6) {
    struct sockaddr_in6 *sin6;

    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
        continue;
}
```

“Test case”:

```

```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Snippet under test:

```
/* Skip IPv6 link-local addresses */
if (family == AF_INET6) {
    struct sockaddr_in6 *sin6;

    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
        continue;
}
```

https://sources.debian.org/src/hostname/3.23%2Bnmu1/hostname.c#L311

“Test case”:

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Snippet under test:

```
/* Skip IPv6 link-local addresses */
if (family == AF_INET6) {
    struct sockaddr_in6 *sin6;

    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
        continue;
}
```

https://sources.debian.org/src/hostname/3.23%2Bnmu1/hostname.c#L311

“Test case”:

```
$ hostname
My-Ubuntu-Laptop
```

In Reality, Coverage Tools Can Be Wild…

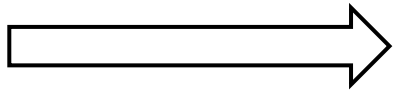
An LLVM-cov Bug Found in Our Work

Snippet under test:

```
/* Skip IPv6 link-local addresses */
if (family == AF_INET6) {
    struct sockaddr_in6 *sin6;

    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
        continue;
}
```

https://sources.debian.org/src/hostname/3.23%2Bnmu1/hostname.c#L311

Coverage tools



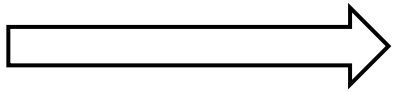
“Test case”:

```
$ hostname
My-Ubuntu-Laptop
```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools



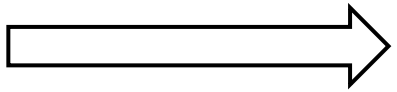
Coverage report for `hostname.c` by LLVM-cov

```
/* Skip IPv6 link-local addresses */
if (family == AF_INET6) {
    struct sockaddr_in6 *sin6;

    sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
    if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
        IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
        continue;
}
```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools



Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
0 |          continue;
0 |  }
```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools

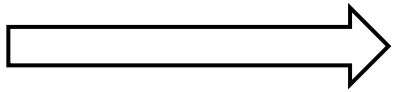
Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  |    [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  |
0 |          continue;
0 |  }
```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools



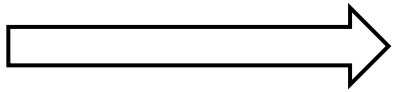
Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  |    [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  |    [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools



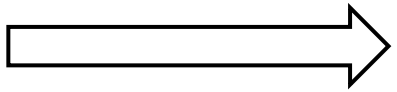
Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools



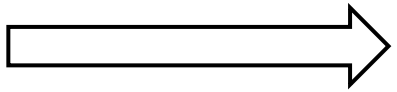
Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  |    [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  |    [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```

In Reality, Coverage Tools Can Be Wild…

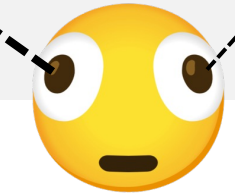
An LLVM-cov Bug Found in Our Work

Coverage tools



Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```



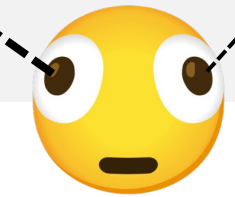
In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools

Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```



$1.84\text{E} = 1.84\times10^{18}$

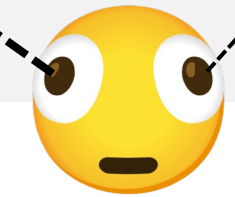
In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools

Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```



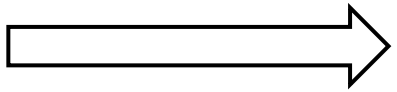
$1.84\text{E} = 1.84\times10^{18}$

An Intel Core i9-14900k (5 GHz) would need **117 years** to print your hostname!

In Reality, Coverage Tools Can Be Wild…

An LLVM-cov Bug Found in Our Work

Coverage tools



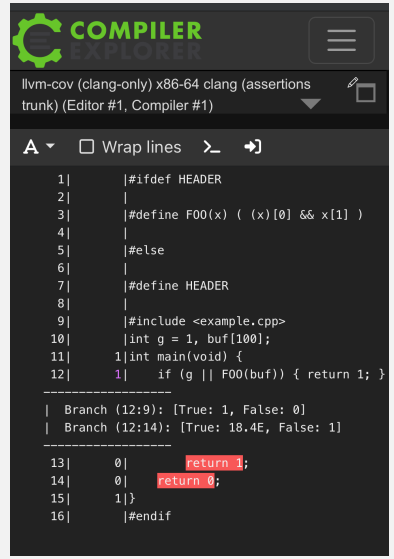
Coverage report for `hostname.c` by LLVM-cov

```
  |  /* Skip IPv6 link-local addresses */
0 |  if (family == AF_INET6) {
0 |      struct sockaddr_in6 *sin6;
  |
0 |      sin6 = (struct sockaddr_in6 *)ifap->ifa_addr;
0 |      if (IN6_IS_ADDR_LINKLOCAL(&sin6->sin6_addr) ||
  [True: 0, False: 0]
0 |          IN6_IS_ADDR_MC_LINKLOCAL(&sin6->sin6_addr))
  [True: 18446744073709551615, False: 1]
0 |          continue;
0 |  }
```

Deeper into This LLVM-cov Bug

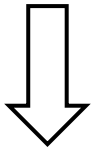
① Original `hostname.c` of ~500 LoC

You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4



Deeper into This LLVM-cov Bug

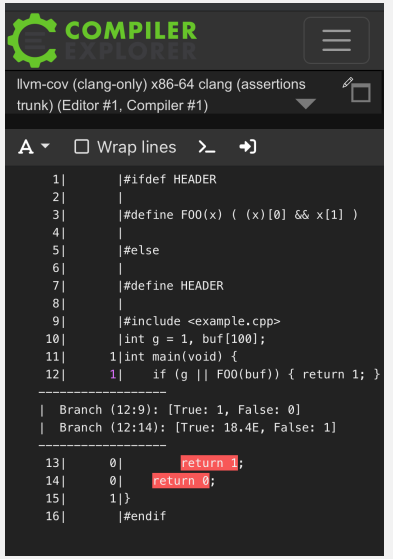
① Original `hostname.c` of ~500 LoC



② Smaller program of 7 LoC retaining the symptom

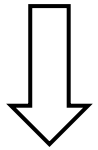
You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4

COMPILER EXPLORER



Deeper into This LLVM-cov Bug

① Original `hostname.c` of ~500 LoC



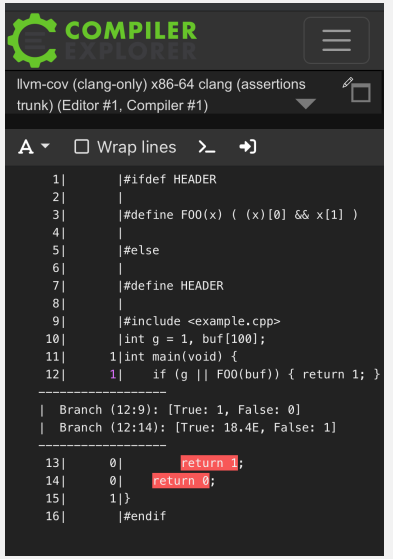
② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define FOO(x) ((x)[0] && x[1])
```

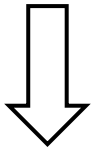
You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4

COMPILER EXPLORER



Deeper into This LLVM-cov Bug

① Original `hostname.c` of ~500 LoC



② Smaller program of 7 LoC retaining the symptom

`test.h`

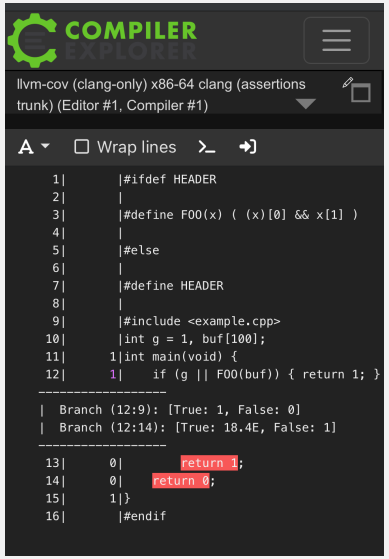
```
#define FOO(x) ((x)[0] && x[1])
```

`test.c`

```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || FOO(buf)) { return 1; }
    return 0;
}
```

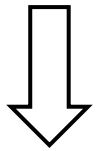
You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4

COMPILER EXPLORER



Deeper into This LLVM-cov Bug

① Original `hostname.c` of ~500 LoC



② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define FOO(x) ((x)[0] && x[1])
```

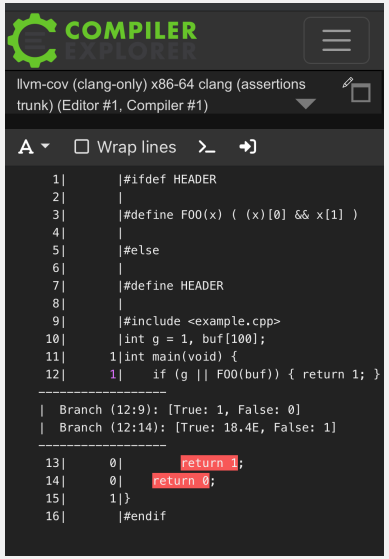
`test.c`

```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || FOO(buf)) { return 1; }
    return 0;
}
```

1. a macro from system header

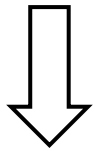
You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4

COMPILER EXPLORER



Deeper into This LLVM-cov Bug

① Original `hostname.c` of ~500 LoC



② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define FOO(x) ((x)[0] && x[1])
```

`test.c`

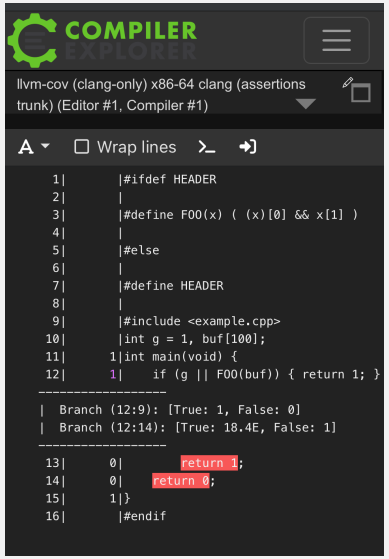
```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || FOO(buf)) { return 1; }
    return 0;
}
```

2. the macro subscripts the argument in parentheses

1. a macro from system header

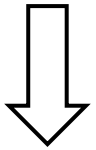
You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4

COMPILER EXPLORER



Deeper into This LLVM-cov Bug

① Original `hostname.c` of ~500 LoC



② Smaller program of 7 LoC retaining the symptom

`test.h`

```
#define FOO(x) ((x)[0] && x[1])
```

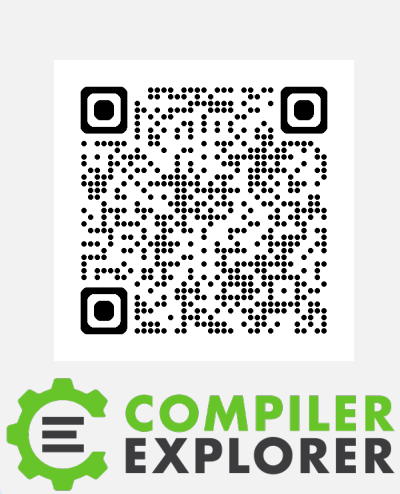
`test.c`

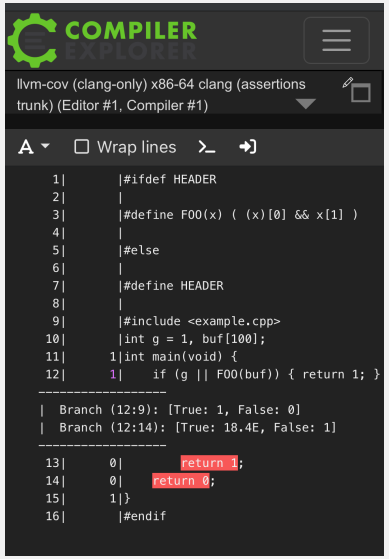
```
#include <test.h>
int g = 1, buf[100];
int main(void) {
    if (g || FOO(buf)) { return 1; }
    return 0;
}
```

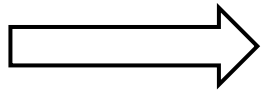
2. the macro subscripts the argument in parentheses

1. a macro from system header

You can play with it on your phone!
https://godbolt.org/z/7W3G61zT4







③ Reported as LLVM#131505 by our work

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
  - **Random programs** generated by Csmith used as test inputs

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
  - **Random programs** generated by Csmith used as test inputs

```
l_1841 ^=
    (l_1630[g_269] <
     (l_1630[(g_176.f3 + 2)] ||
      (((safe_mod_func_int32_t_s_s(
            ((*****g_554) = (0x8CFEFE1AL | (*l_1664))),
            (0x589BL |
             ((safe_unary_minus_func_int64_t_s(
                  (((safe_div_func_int32_t_s_s(((void *)0 != l_1836),
                                               p_28)) ^
                    ((**l_1623) =
                         (((**g_165) = (safe_rshift_func_int16_t_s_s(
                                0x1B78L, 5))) > p_28))),
                   (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
       p_28));
```

A relatively “small” statement generated by Csmith

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
  - **Random programs** generated by Csmith used as test inputs
  - Output: lines where Gcov and LLVM-cov report different line coverage

```
l_1841 ^=
    (l_1630[g_269] <
     (l_1630[(g_176.f3 + 2)] ||
      (((safe_mod_func_int32_t_s_s(
            ((*****g_554) = (0x8CFEFE1AL | (*l_1664))),
            (0x589BL |
             ((safe_unary_minus_func_int64_t_s(
                  (((safe_div_func_int32_t_s_s(((void *)0 != l_1836),
                                               p_28)) ^
                    ((**l_1623) =
                         (((**g_165) = (safe_rshift_func_int16_t_s_s(
                               0x1B78L, 5))) > p_28))),
                   (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
       p_28));
```

A relatively “small” statement generated by Csmith

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
  - **Random programs** generated by Csmith used as test inputs
  - Output: lines where Gcov and LLVM-cov report different line coverage
- Found lots of bugs but still missed the bug exposed with `hostname.c`

```
l_1841 ^=
    (l_1630[g_269] <
     (l_1630[(g_176.f3 + 2)] ||
      (((safe_mod_func_int32_t_s_s(
            ((*****g_554) = (0x8CFEFE1AL | (*l_1664))),
            (0x589BL |
             ((safe_unary_minus_func_int64_t_s(
                  (((safe_div_func_int32_t_s_s(((void *)0 != l_1836),
                                               p_28)) ^
                    ((**l_1623) =
                         (((**g_165) = (safe_rshift_func_int16_t_s_s(
                                0x1B78L, 5))) > p_28))),
                   (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
       p_28));
```

A relatively “small” statement generated by Csmith

Main Prior Work (“C2V”)

- [Yang et al. 2019] proposed **differential testing** of Gcov and LLVM-cov, the most widely used coverage tools for C/C++
  - **Random programs** generated by Csmith used as test inputs
  - Output: lines where Gcov and LLVM-cov report different line coverage
- Found lots of bugs but still missed the bug exposed with `hostname.c`

```
l_1841 ^=
    (l_1630[g_269] <
     (l_1630[(g_176.f3 + 2)] ||
      (((safe_mod_func_int32_t_s_s(
            ((*****g_554) = (0x8CFEFE1AL | (*l_1664))),
            (0x589BL |
             ((safe_unary_minus_func_int64_t_s(
                  (((safe_div_func_int32_t_s_s(((v
                                                 p_2
                    ((**l_1623) =
                         (((**g_165) = (safe_rsh
                               0x1B78L, 5))) > p_2
                     (-1L)))) <= l_1839)))) <= 18446744073709551613UL) !=
       p_28));
```

Though complex, not capturing **real-world** code patterns

A relatively “small” statement generated by Csmith

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**

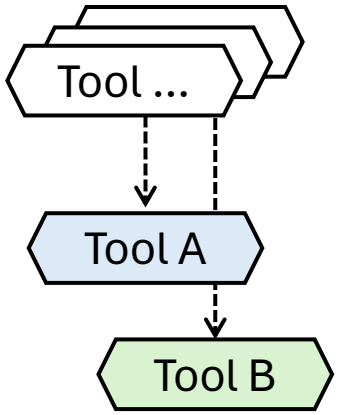
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

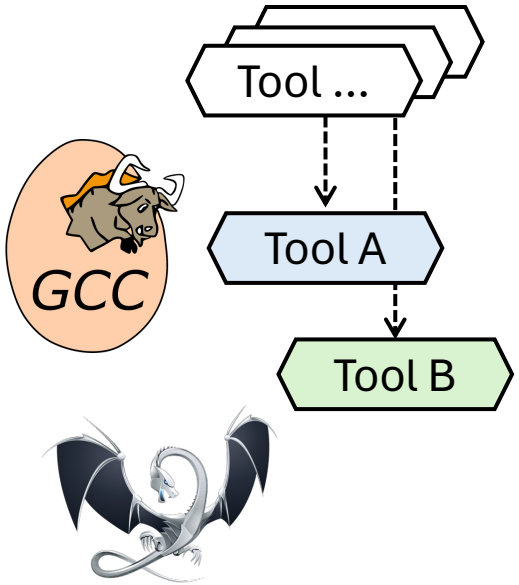
Study Subjects



DEBCOVDIFF: Contributions

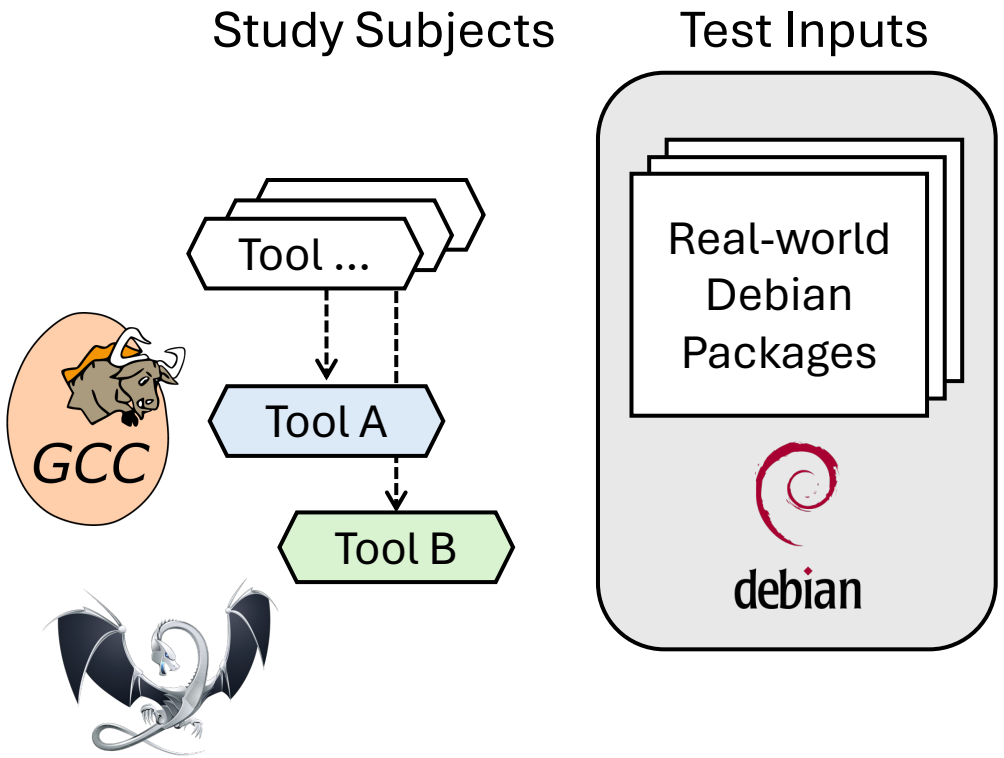
1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**

Study Subjects



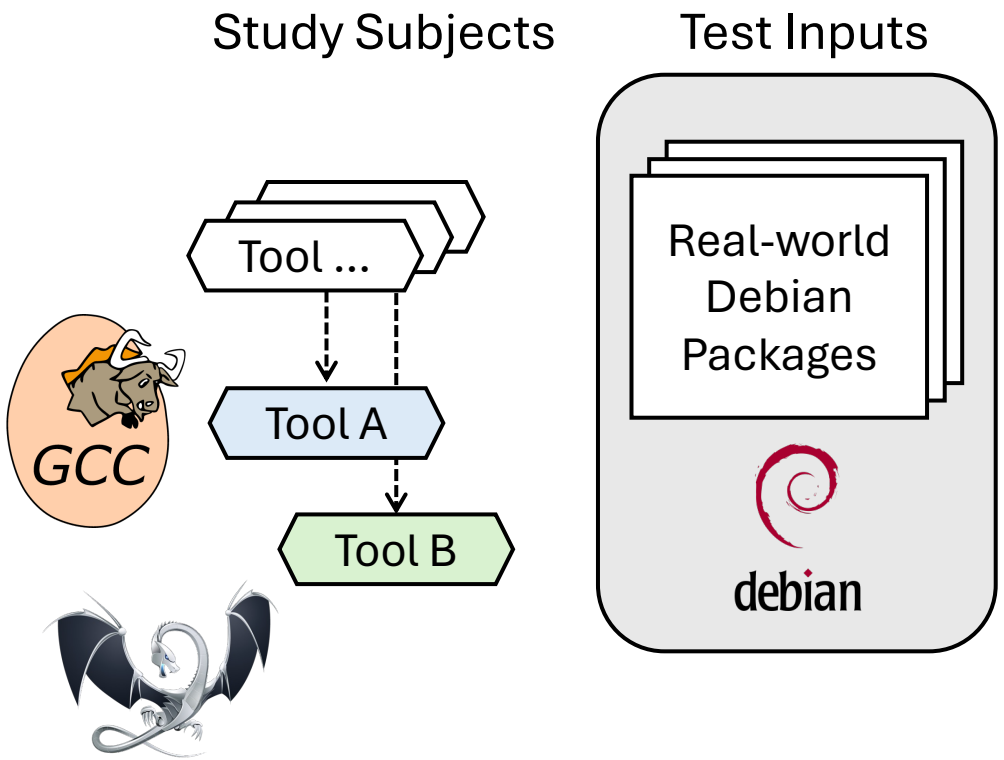
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



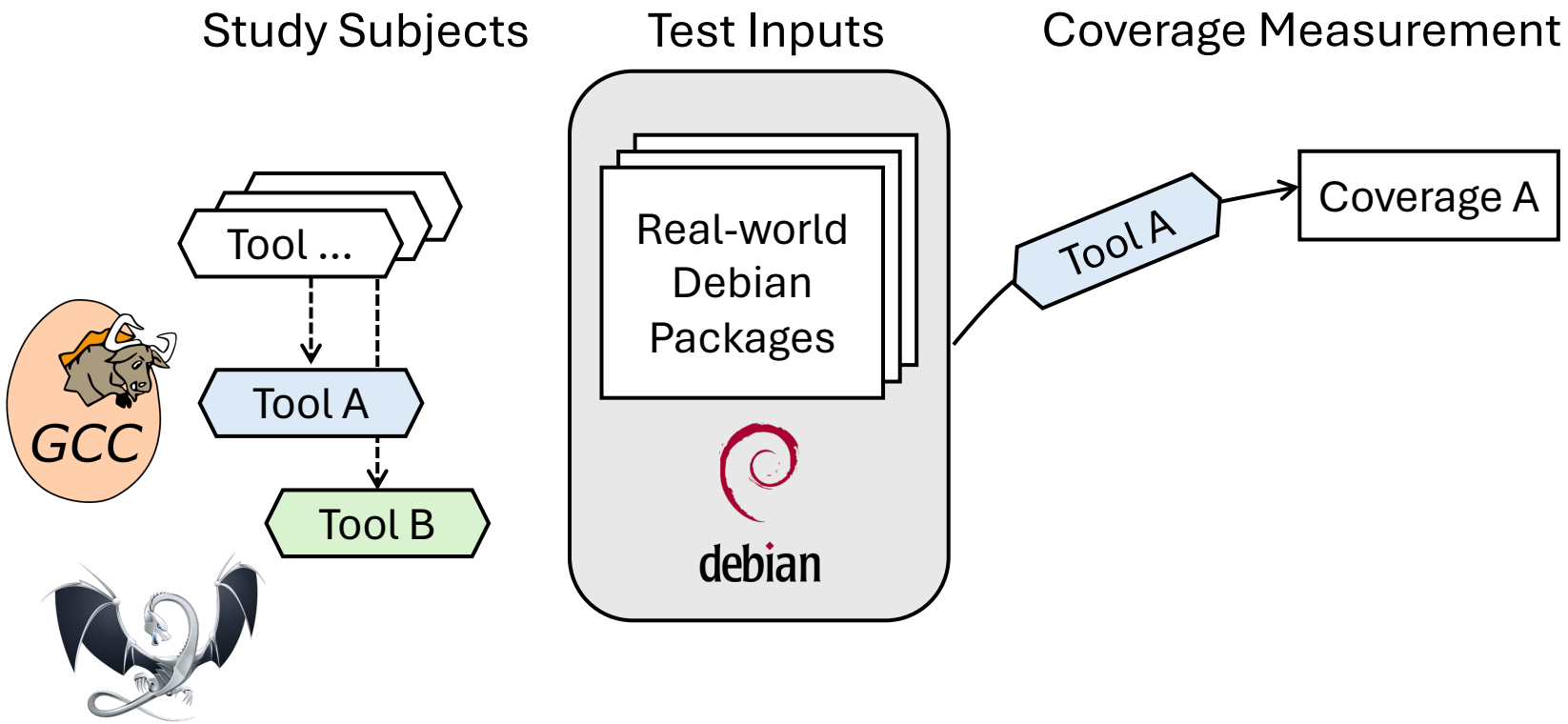
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



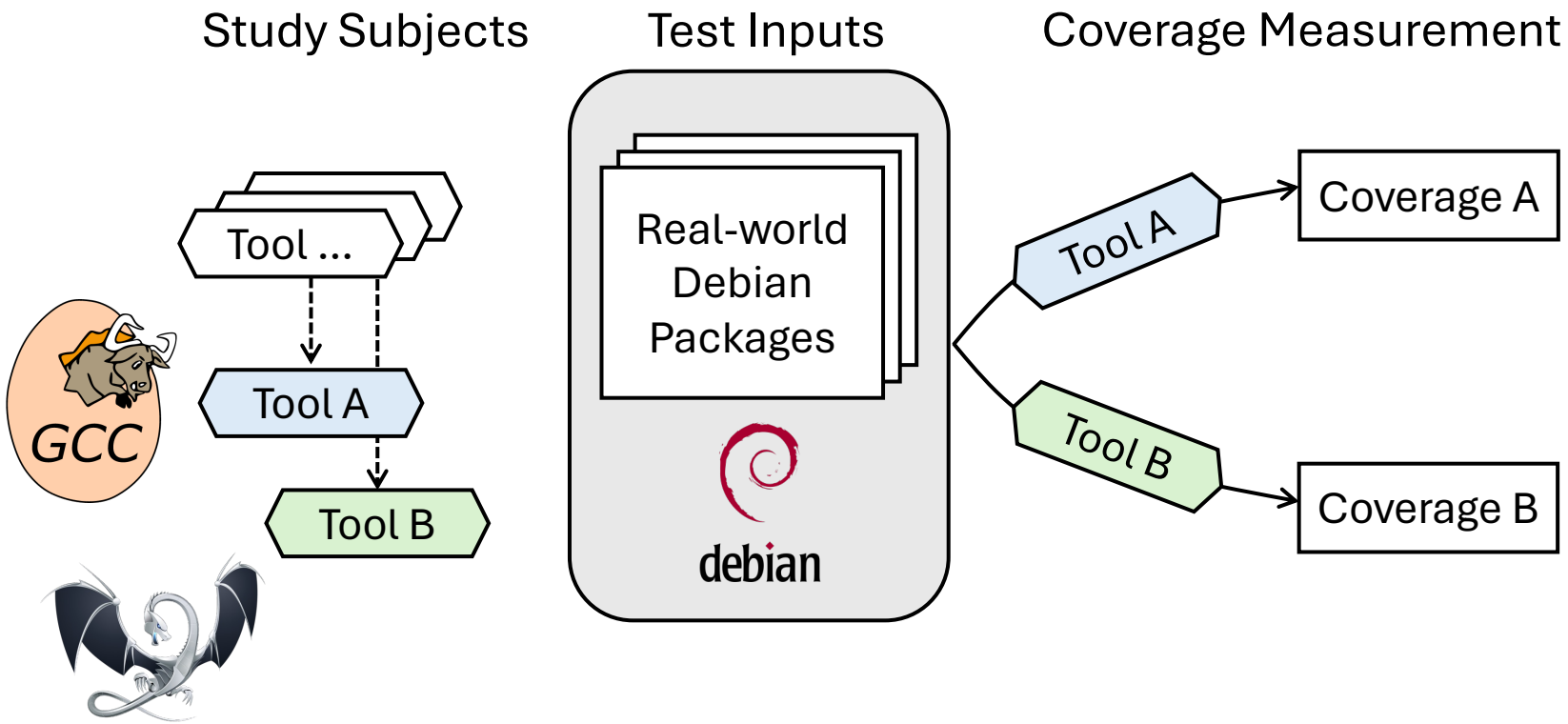
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



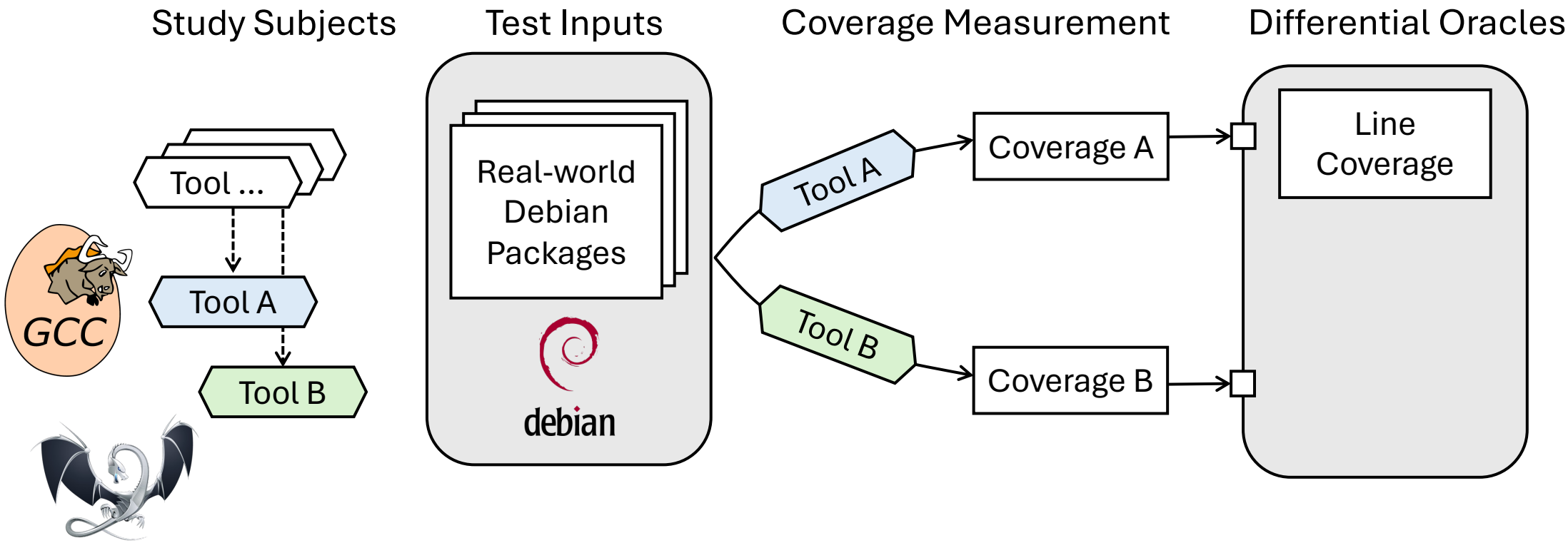
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



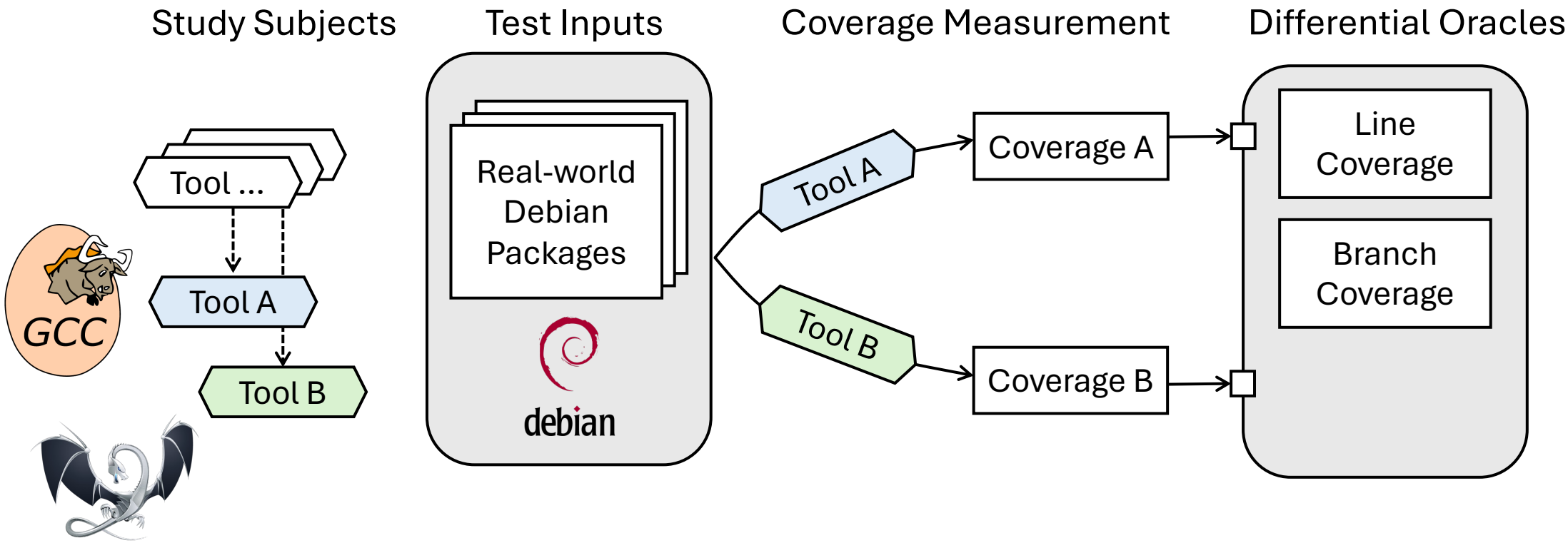
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



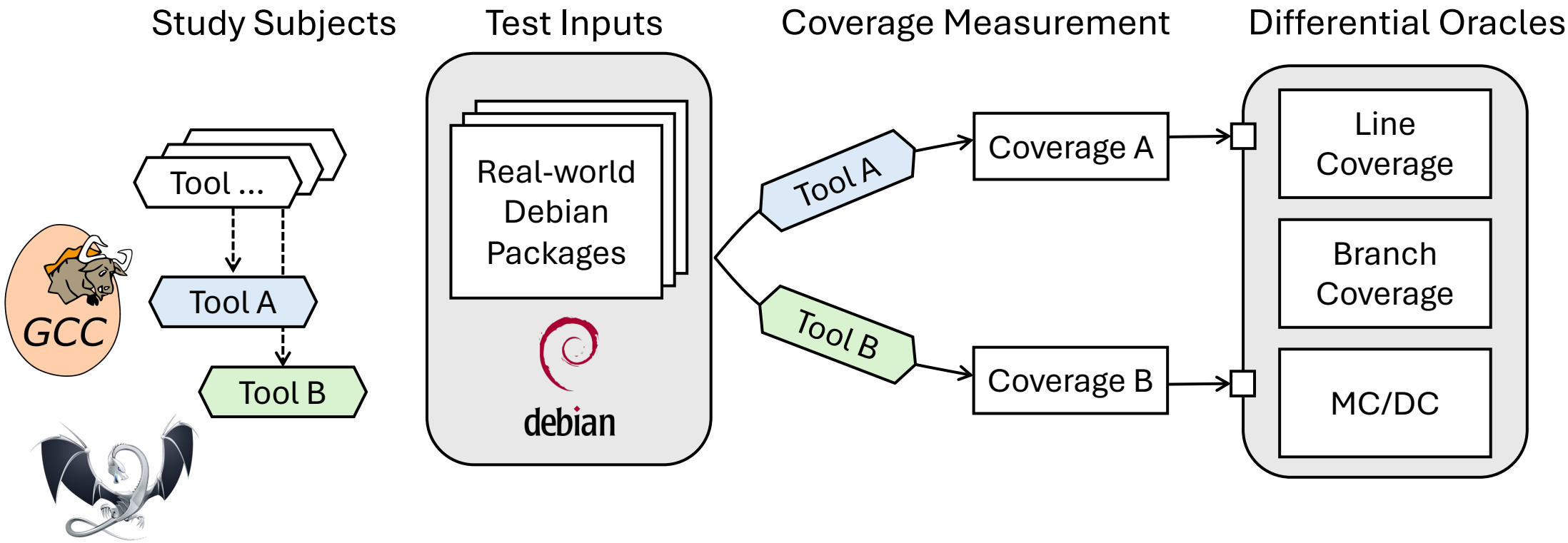
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



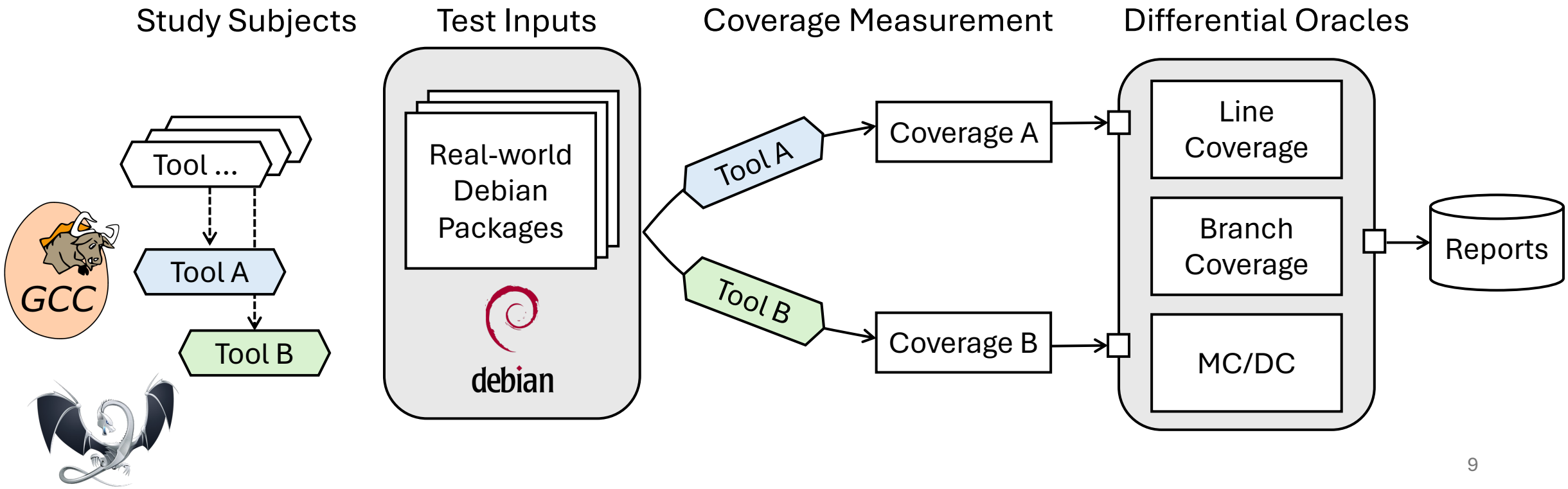
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



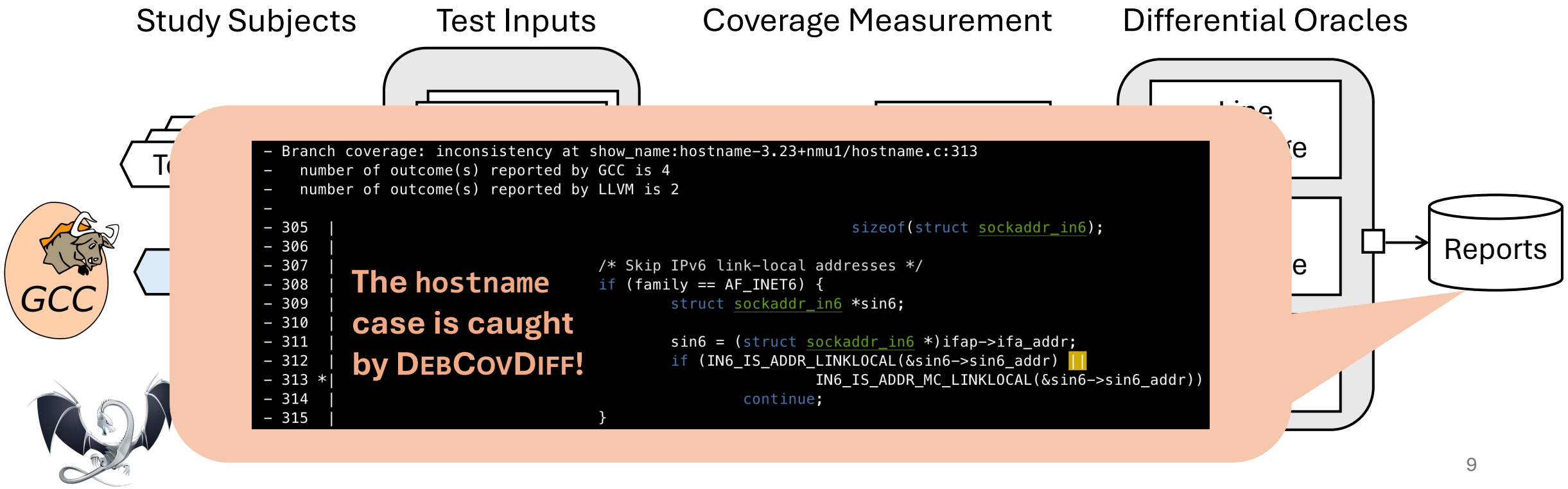
DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



DEBCOVDIFF: Contributions

1. Key insight: **real-world software** used as test inputs for coverage tools
   - Classic **differential testing** as in prior work C2V
2. Mitigate program execution **nondeterminism**
3. Include advanced metrics, i.e., **branch coverage** and **MC/DC**



Debian Packages as Test Input

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

⇩

Priority label being one of **“required”**, **“important”**, and **“standard”**

103

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

⇩

Priority label being one of **“required”**, **“important”**, and **“standard”**

103

⇩

Map binary packages to **source packages**

78

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

⇩

Priority label being one of **“required”**, **“important”**, and **“standard”**

103

⇩

Map binary packages to **source packages**

78

⇩

Primary programming language is **C/C++**

57

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

⇩

Priority label being one of **“required”**, **“important”**, and **“standard”**

103

⇩

Map binary packages to **source packages**

78

⇩

Primary programming language is **C/C++**

57

⇩

We are able to **measure coverage** using both Gcov and LLVM-cov successfully

32

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

⇩

Priority label being one of **“required”**, **“important”**, and **“standard”**

103

⇩

Map binary packages to **source packages**

78

⇩

Primary programming language is **C/C++**

57

⇩

We are able to **measure coverage** using both Gcov and LLVM-cov successfully

32

⇩

Randomly 10 more, 9 of which produce coverage successfully

41

Selection criteria:
relevance and heterogeneity

Debian Packages as Test Input

All Debian 12.8 (“bookworm”) binary packages for **amd64** under **“main”** component

63,417

⇩

Priority label being one of **“required”**, **“important”**, and **“standard”**

103

⇩

Map binary packages to **source packages**

78

⇩

Primary programming language is **C/C++**

57

⇩

We are able to **measure coverage** using both Gcov and LLVM-cov successfully

32

⇩

Randomly 10 more, 9 of which produce coverage successfully

41

Selection criteria:
relevance and heterogeneity

Package examples

- Essential system utilities: **`coreutils`**
- Networking tools: **`iproute2`**
- Compression libraries: **`bzip2`**
- Services & management: **`apache2`**
- User-facing tools: **`nano`**

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

```
$ hostname
My-Ubuntu-Laptop
```

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

```
$ hostname
My-Ubuntu-Laptop
```

   - Run with all **41** packages

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

```
$ hostname
My-Ubuntu-Laptop
```

   - Run with all **41** packages

2. Existing Tests (ET)

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

```
$ hostname
My-Ubuntu-Laptop
```

   - Run with all **41** packages

2. Existing Tests (ET)
   - `dh_auto_test` (part of the Debian infrastructure)

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

```
$ hostname
My-Ubuntu-Laptop
```

   - Run with all **41** packages

2. Existing Tests (ET)
   - `dh_auto_test` (part of the Debian infrastructure)
   - Automatically probe targets such as “check” and “test” in `Makefile`, `build.ninja` etc.

Tests on Debian

Goal: execute *some* code paths in Debian packages so that coverage comparison is more meaningful

1. Simple Commands (SC)

```
$ hostname
My-Ubuntu-Laptop
```

   - Run with all **41** packages

2. Existing Tests (ET)
   - `dh_auto_test` (part of the Debian infrastructure)
   - Automatically probe targets such as “check” and “test” in `Makefile`, `build.ninja` etc.
   - Run with **9** packages

Differential Oracles

- Line coverage

```
3 |
3 | (C/C++ source line...)
3 |
```

Gcov report

```
3 |
6 | (C/C++ source line...)
3 |
```

LLVM-cov report

Differential Oracles

- Line coverage

```
✓ 3 |
  3 | (C/C++ source line...)
  3 |
```

Gcov report

```
✓ 3 |
  6 | (C/C++ source line...)
  3 |
```

LLVM-cov report

Differential Oracles

- Line coverage

```
✓ 3 |
? 3 | (C/C++ source line...)
  3 |
```

Gcov report

```
✓ 3 |
? 6 | (C/C++ source line...)
  3 |
```

LLVM-cov report

Differential Oracles

- Line coverage

```
✓ 3 |
? 3 | (C/C++ source line...)
  3 |
```

Gcov report

```
✓ 3 |
? 6 | (C/C++ source line...)
  3 |
```

LLVM-cov report

Emit: ['line_val', <source_location>, 3, 6]

Differential Oracles

- Line coverage

```
✓ 3 |
? 3 | (C/C++ source line...)
✓ 3 |
```

Gcov report

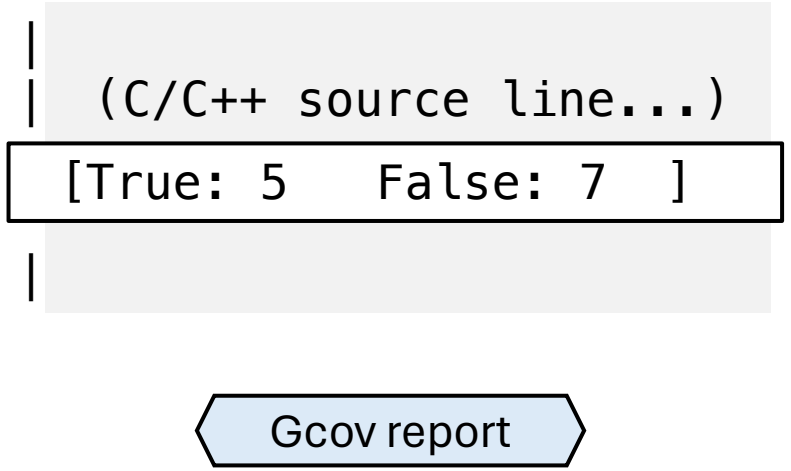
```
✓ 3 |
? 6 | (C/C++ source line...)
✓ 3 |
```

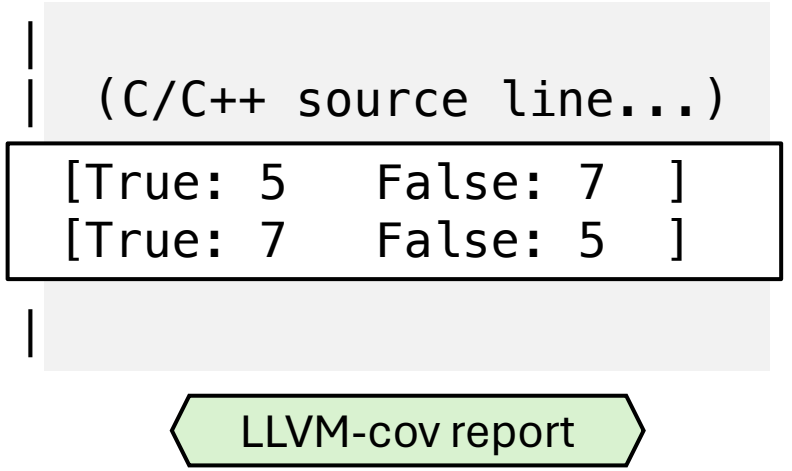
LLVM-cov report

Emit: ['line_val', <source_location>, 3, 6]

Differential Oracles

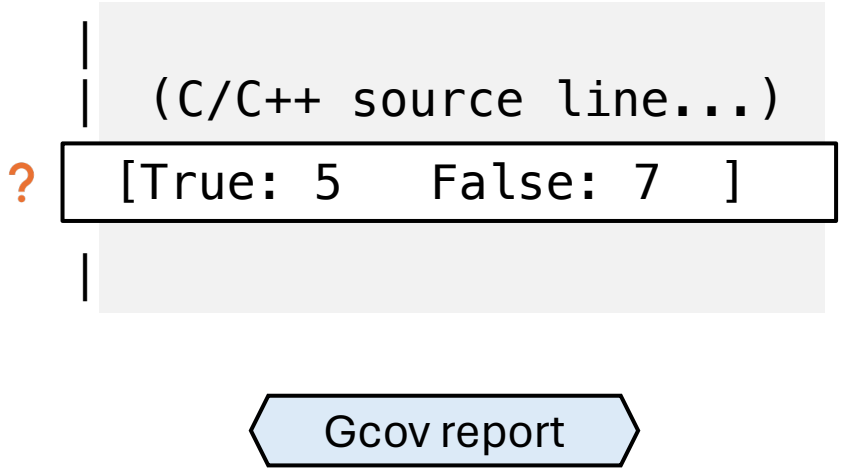
- Branch coverage

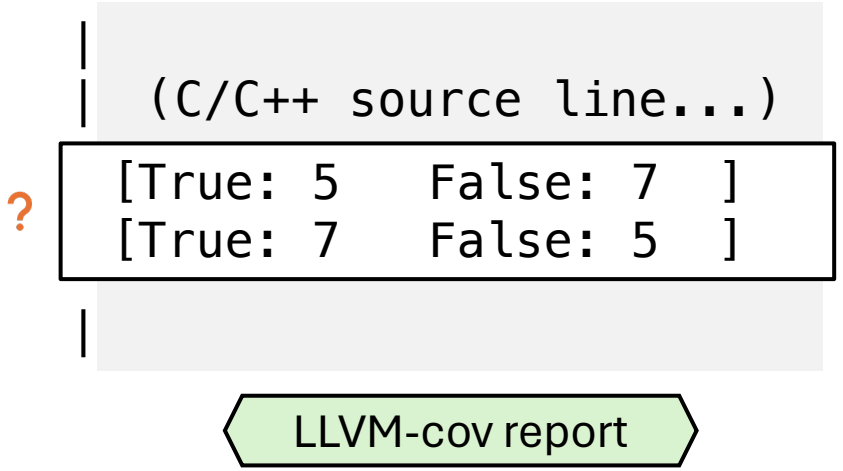




Differential Oracles

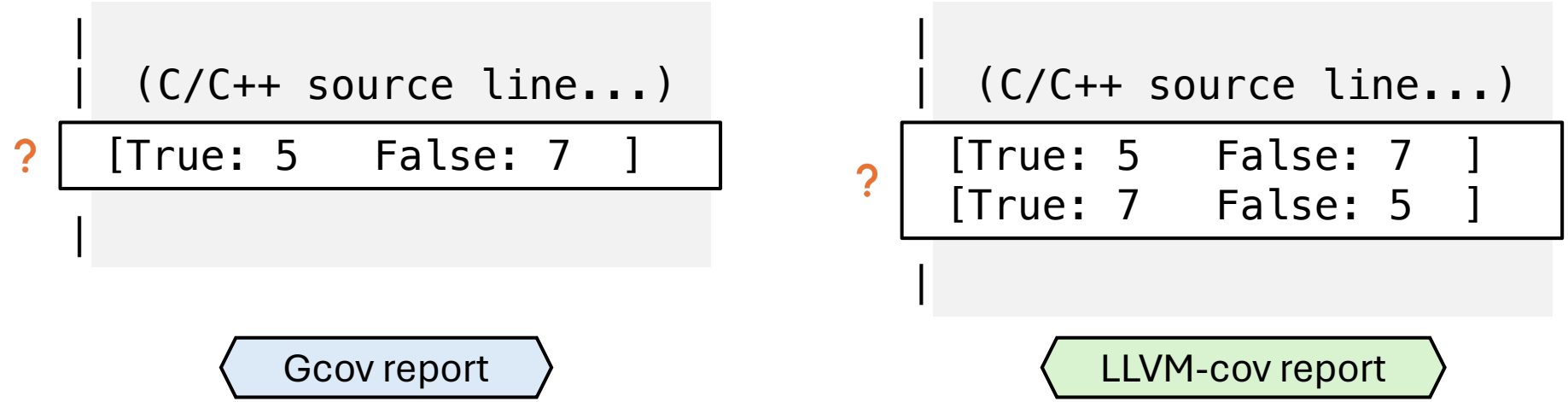
- Branch coverage





Differential Oracles

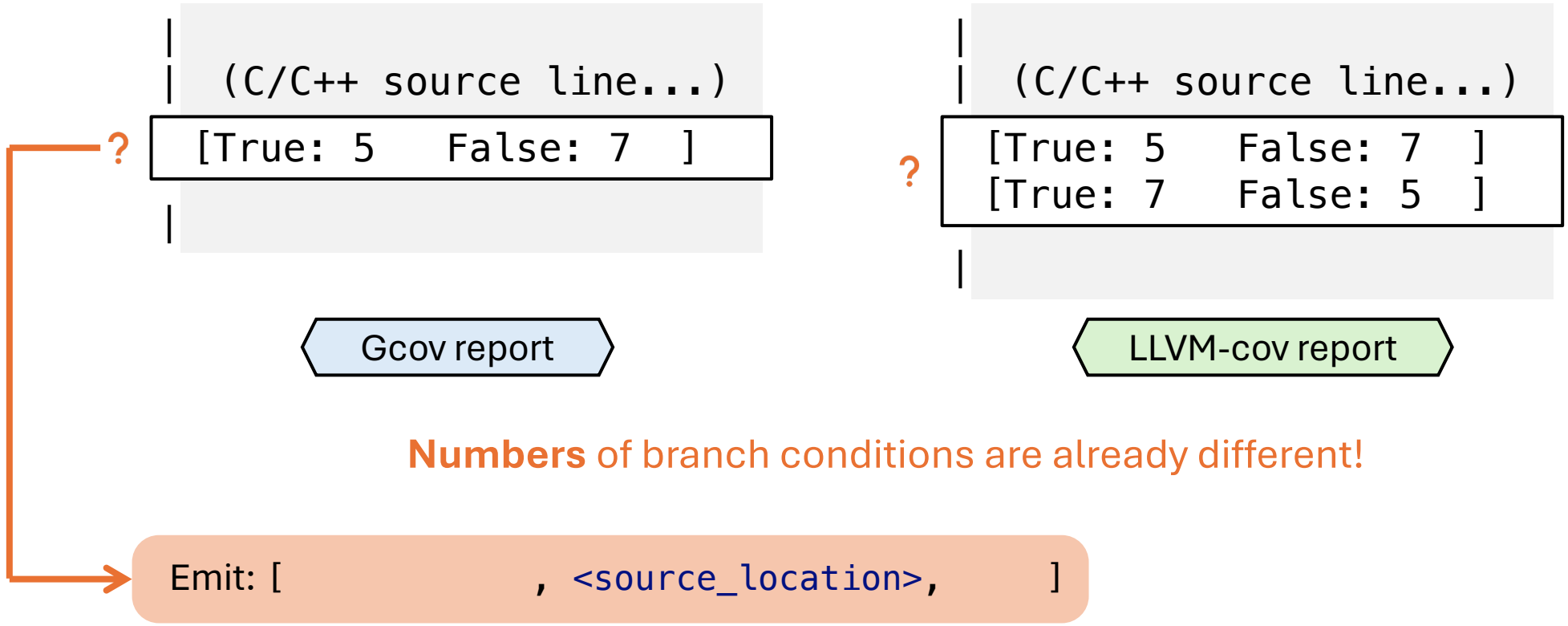
- Branch coverage



Numbers of branch conditions are already different!

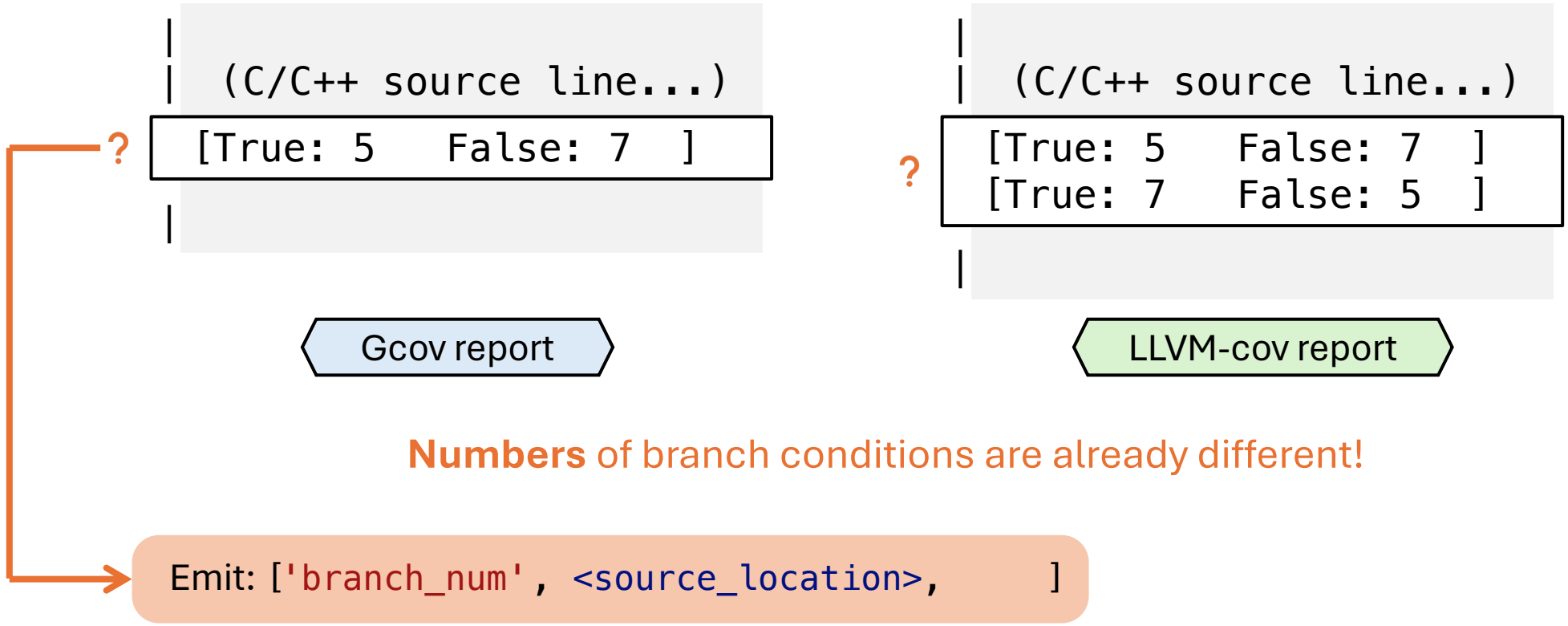
Differential Oracles

- Branch coverage



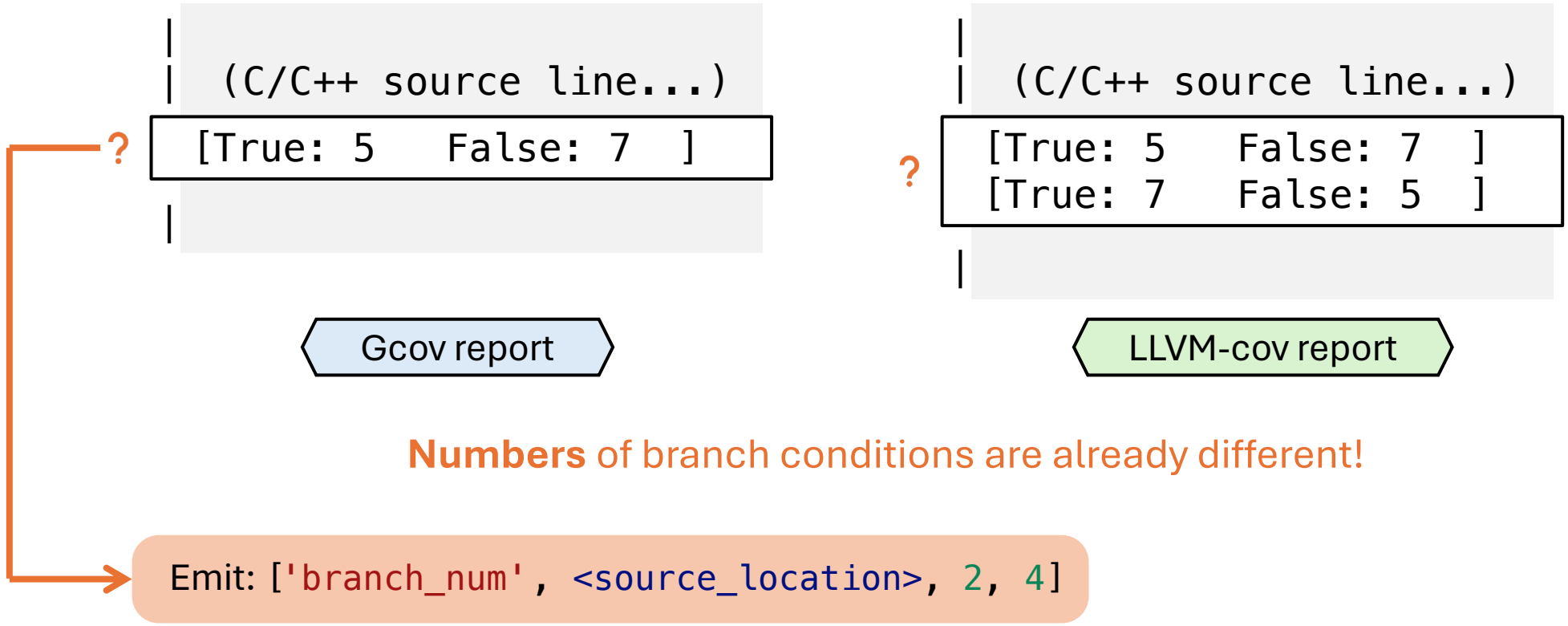
Differential Oracles

- Branch coverage



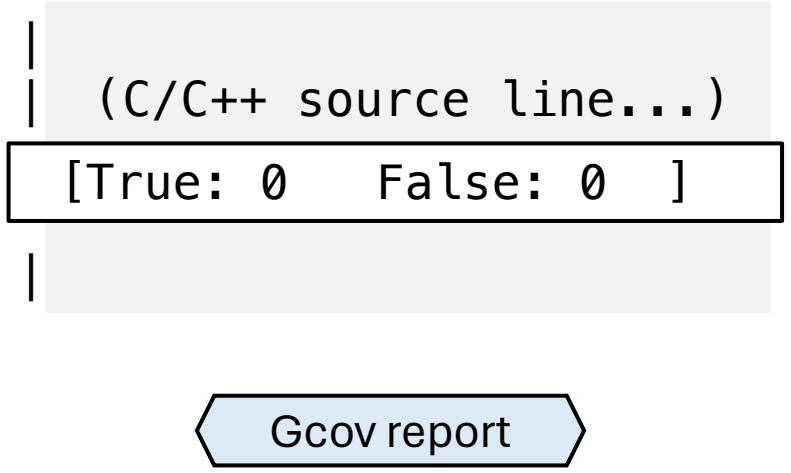
Differential Oracles

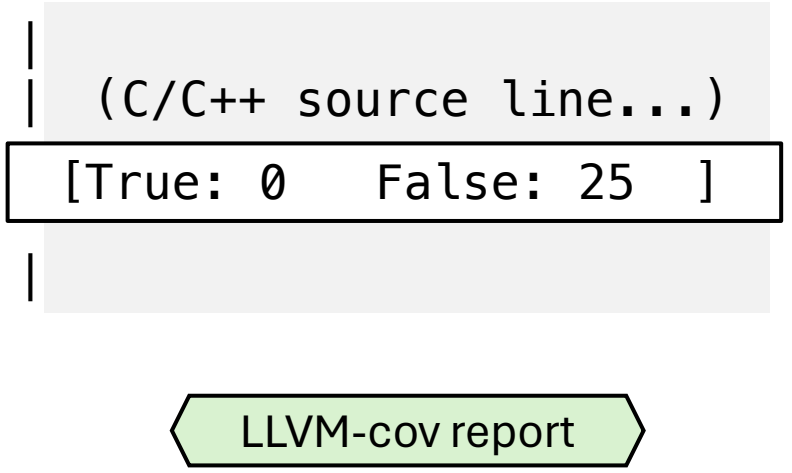
- Branch coverage



Differential Oracles

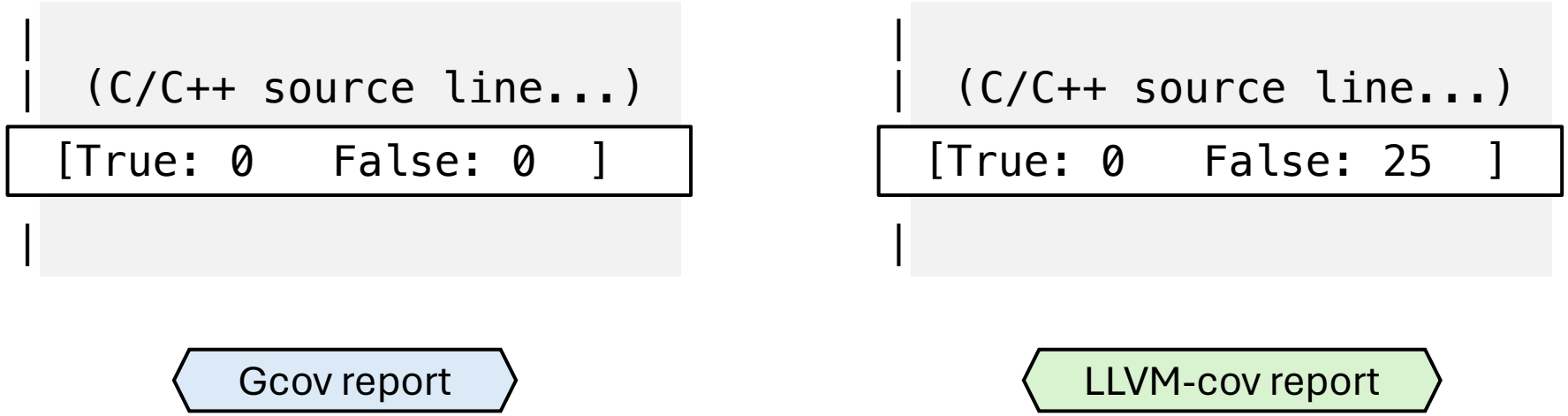
- Branch coverage





Differential Oracles

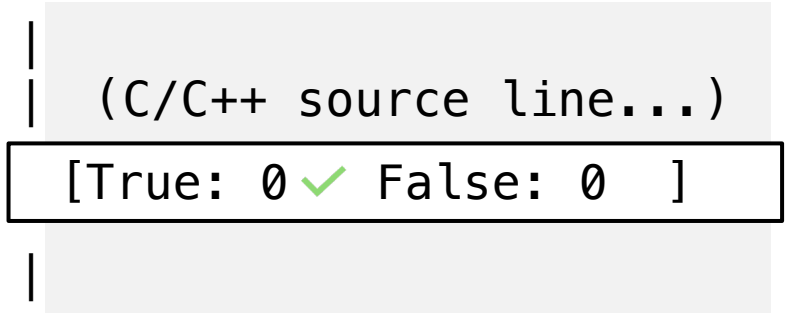
- Branch coverage

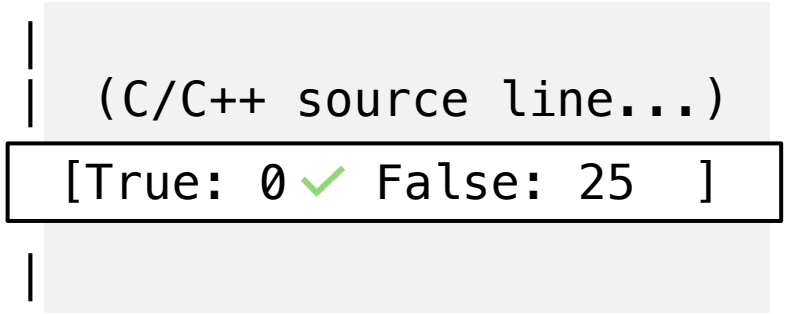


Number of branch conditions are the same: ✓
Move on to **value** comparison

Differential Oracles

- Branch coverage





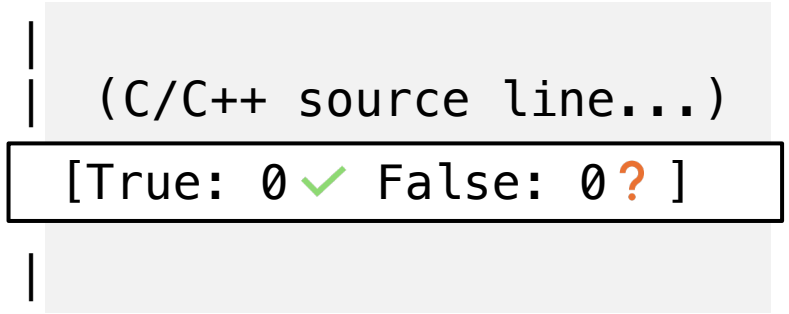
Gcov report

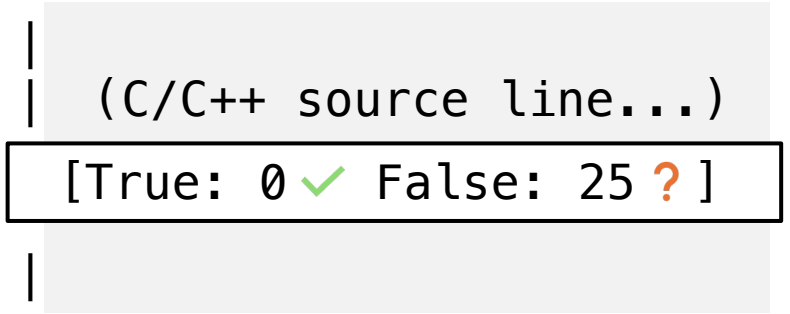
LLVM-cov report

Number of branch conditions are the same: ✓
Move on to **value** comparison

Differential Oracles

- Branch coverage





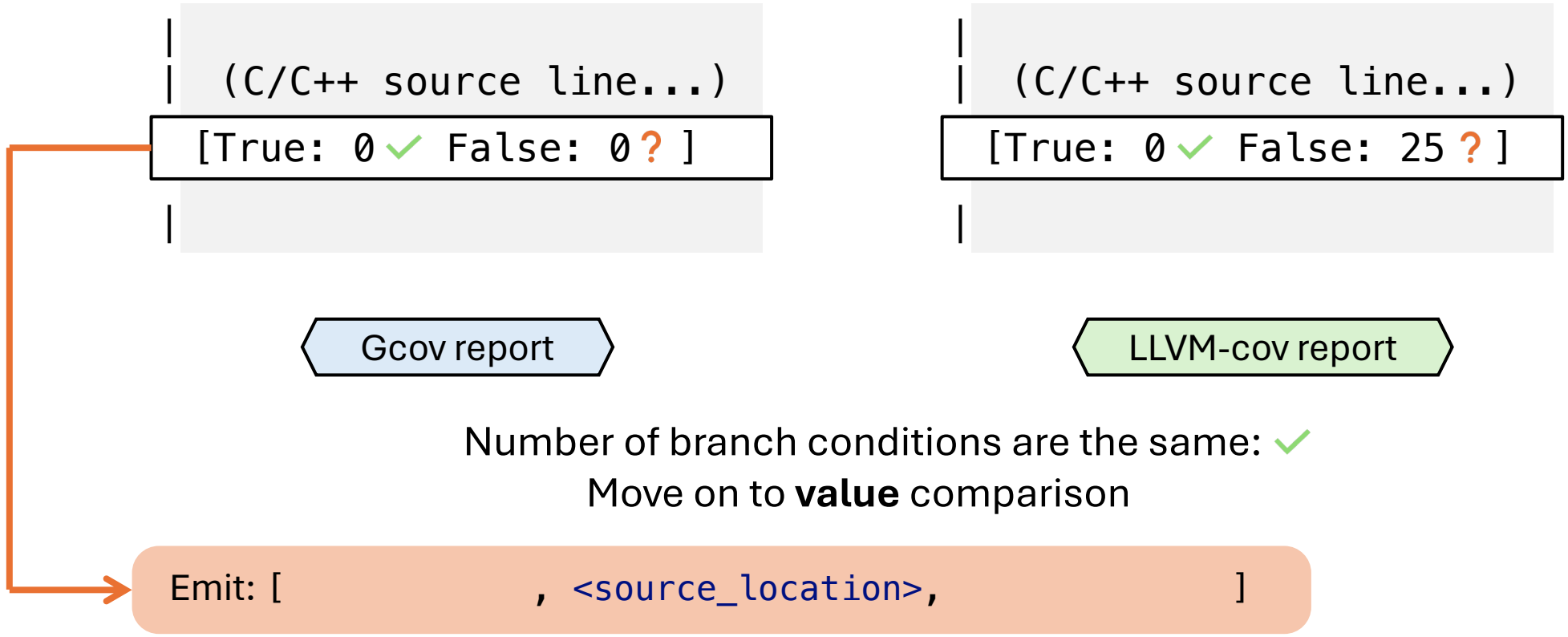
Gcov report

LLVM-cov report

Number of branch conditions are the same: ✓
Move on to **value** comparison

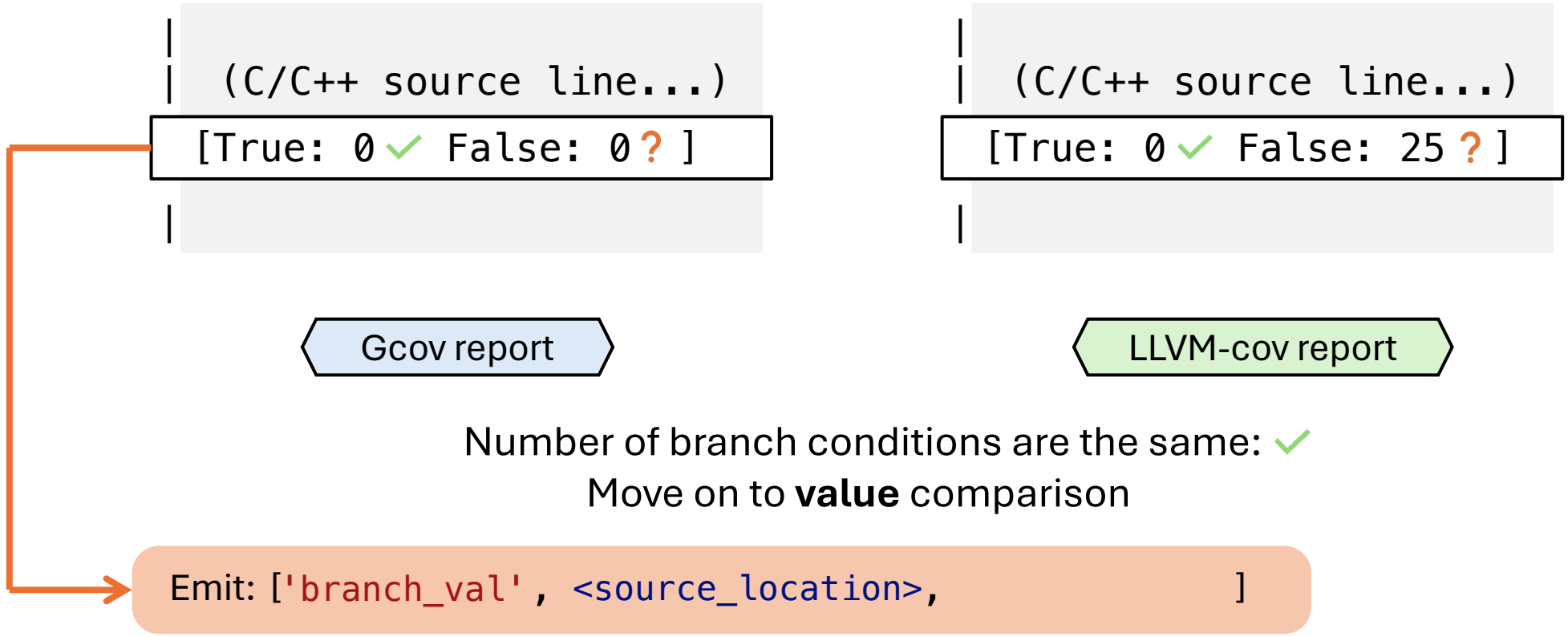
Differential Oracles

- Branch coverage



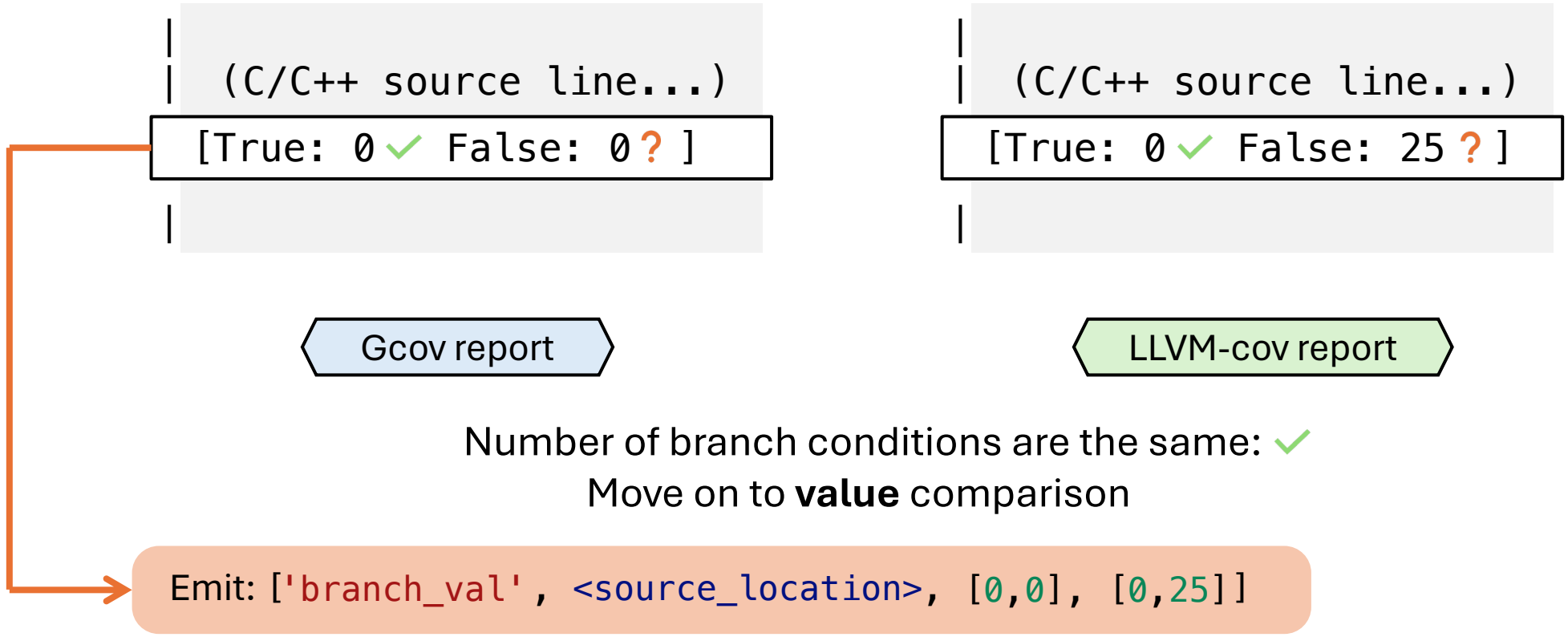
Differential Oracles

- Branch coverage



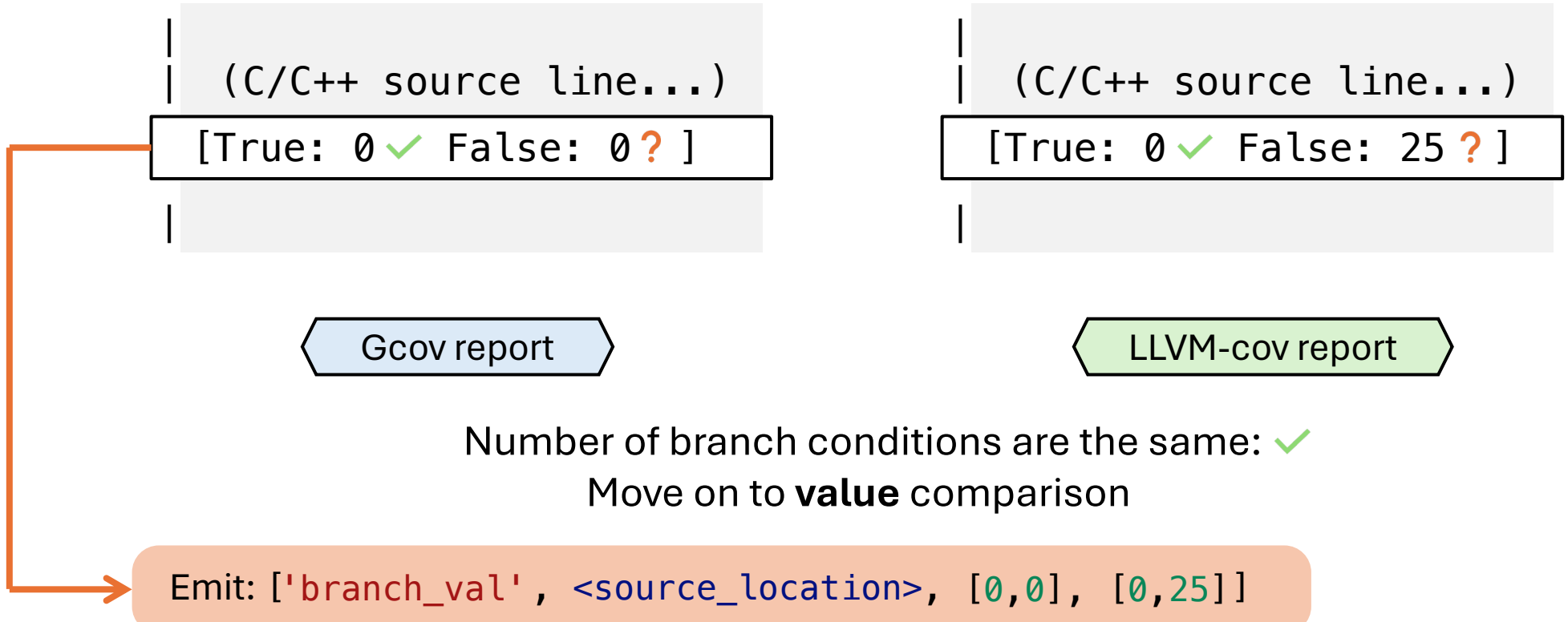
Differential Oracles

- Branch coverage



Differential Oracles

- Branch coverage



- MC/DC: `'mcdc_num'` and `'mcdc_val'`

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)

repeat = 1 `['line_val', 'src/ps/display.c:318', 6779, ..]`

repeat = 2 `['line_val', 'src/ps/display.c:318', 6878, ..]`

⋮

repeat = T `['line_val', 'src/ps/display.c:318', 6812, ..]`

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)

repeat = 1 `['line_val', 'src/ps/display.c:318', 6779, ..]`

repeat = 2 `['line_val', 'src/ps/display.c:318', 6878, ..]`

⋮

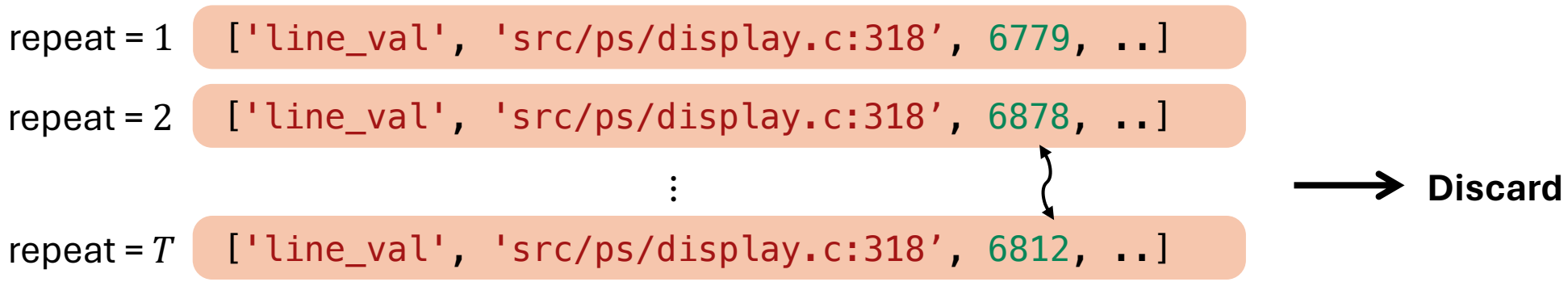
repeat = T `['line_val', 'src/ps/display.c:318', 6812, ..]`

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)



Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)

repeat = 1 `['line_val', 'src/ps/display.c:639', 462, 308]`

repeat = 2 `['line_val', 'src/ps/display.c:639', 462, 308]`

⋮

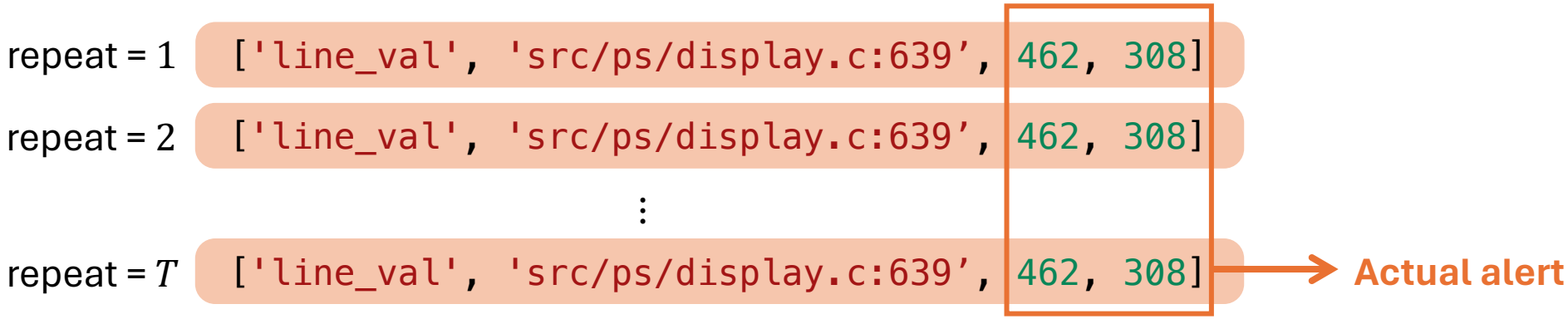
repeat = T `['line_val', 'src/ps/display.c:639', 462, 308]`

Program Nondeterminism

- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)

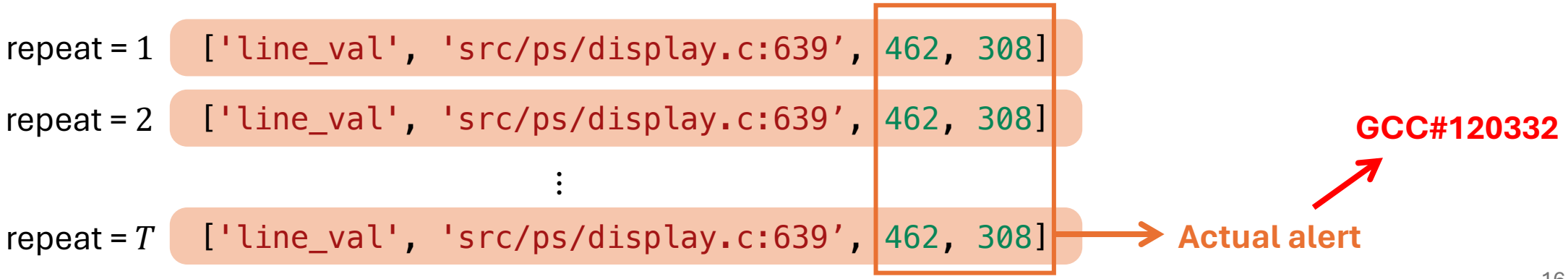


Program Nondeterminism

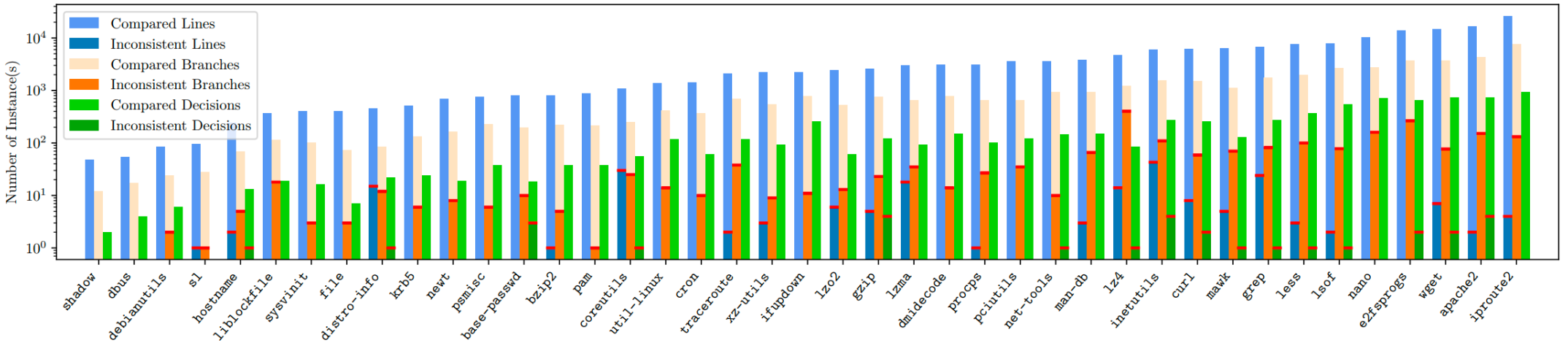
- Even **the same tool can report different coverage** at different time or with different machine environment
  - Largely depending on package and tests

```
$ ps # part of "procps" package
    PID TTY          TIME CMD
2568258 pts/0    00:00:00 bash
2568281 pts/0    00:00:00 ps
```

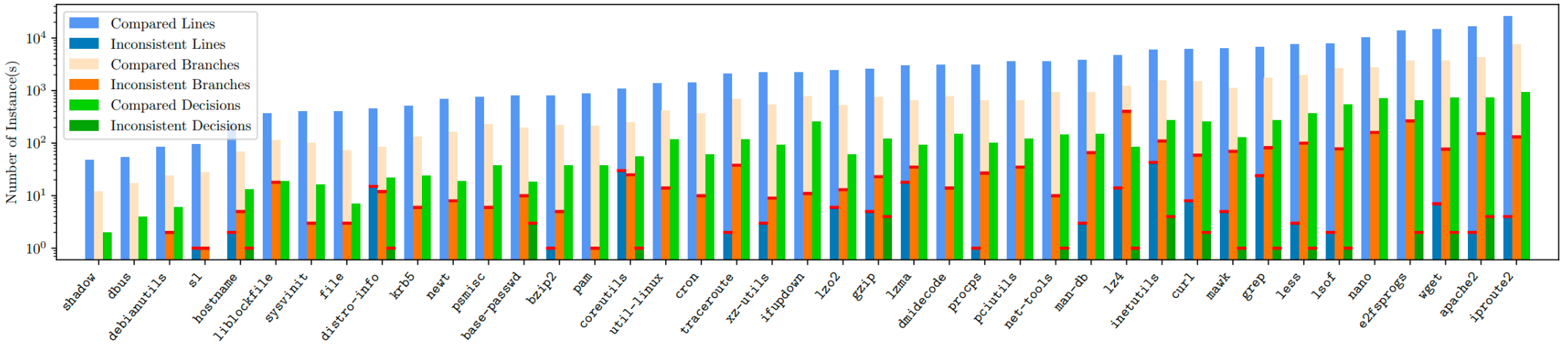
- Countermeasure: hold off the alert until seeing stable inconsistency for repeated T runs ($T = 6$ for SC, 60 for ET)



Inconsistencies in SC

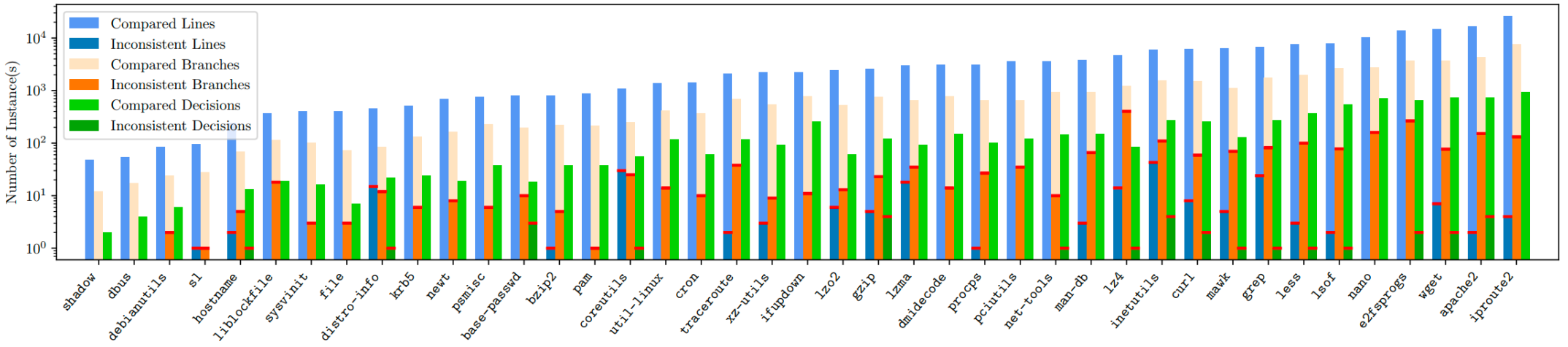


Inconsistencies in SC



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (*_num + *_val)	199	2,098 (1,884 + 214)	30 (29 + 1)

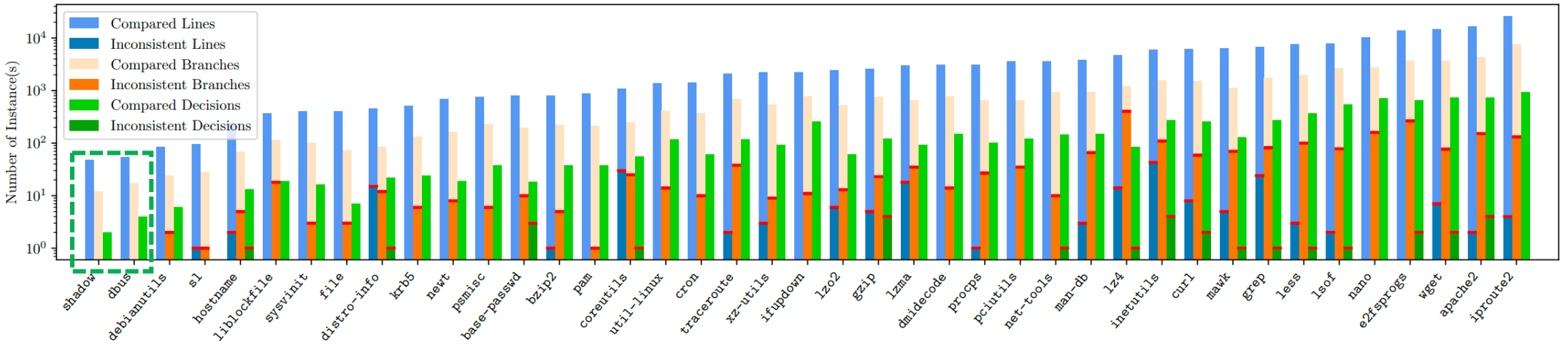
Inconsistencies in SC



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (*_num + *_val)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies

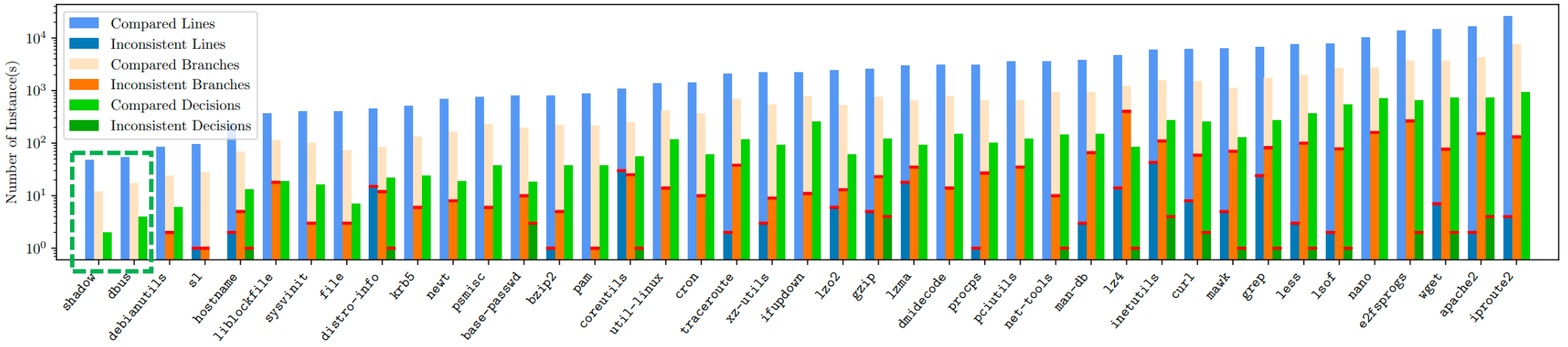
Inconsistencies in SC



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (*_num + *_val)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies

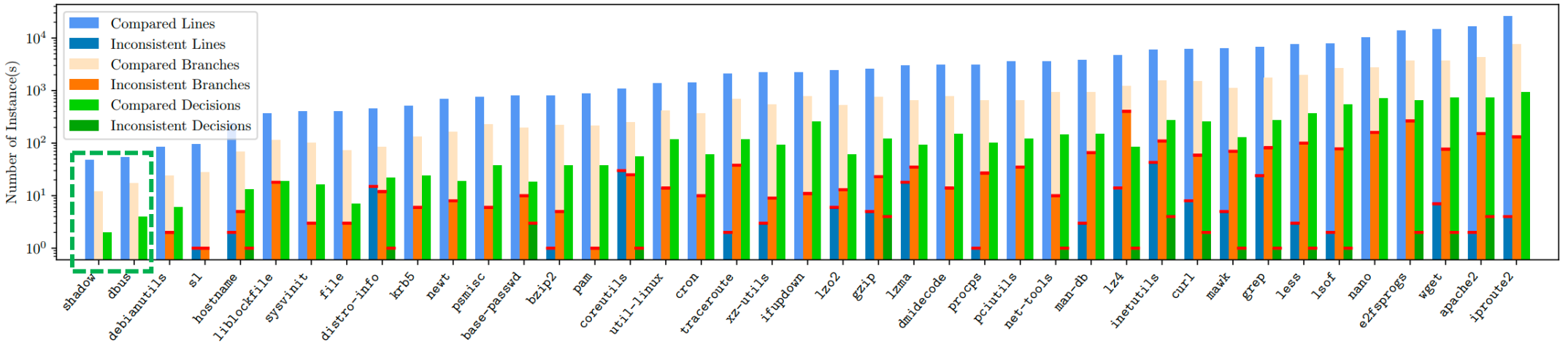
Inconsistencies in SC



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (*_num + *_val)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies
- Sampled **1,240** for inspection

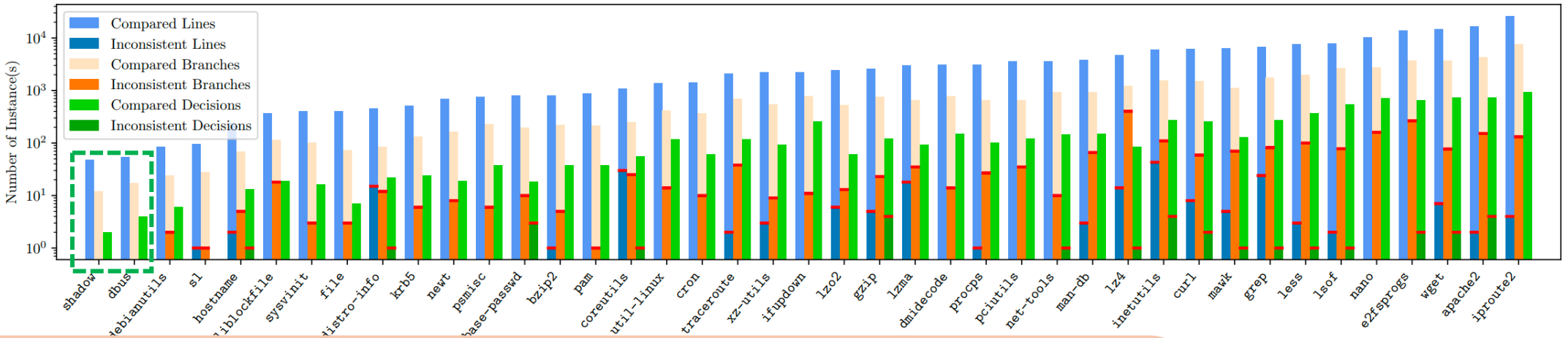
Inconsistencies in SC



	Lines	Branches	MC/DC
Total compared	168,549	44,126	7,527
Total inconsistent (*_num + *_val)	199	2,098 (1,884 + 214)	30 (29 + 1)

- **39** packages exhibit in total **2,327** inconsistencies
- Sampled **1,240** for inspection
  1. Bugs (256, detailed later)
  2. Conventions (954, detailed later)
  3. Compiler-specific differences (30)

Inconsistencies in SC




```
< -I/usr/include/p11-kit-1 -DHAVE_LIBGNUTLS -DNDEBUG -O0 --coverage
< -fcondition-coverage -DNO_SSLv2 -D_FILE_OFFSET_BITS=64 -g -Wall
---
> -I/usr/include/p11-kit-1 -DHAVE_LIBGNUTLS -DNDEBUG -O0
> -fprofile-instr-generate -fcoverage-mapping -fcoverage-mcdc
> -DNO_SSLv2 -D_FILE_OFFSET_BITS=64 -g -Wall
```

(*_num + *_val)	199	(1,884 + 214)	(29 + 1)

exhibit in total **2,327**
ies

40 for inspection

Bugs (256, detailed later)

Conventions (954, detailed later)

3. Compiler-specific differences (30)

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	9 (`lzo2`)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	2 (`lzo2`)	3 + 2	inline function, multistmt
3	117412	Gcov	L	3 (`lz4`)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (`lz4`)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (`lz4`)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (`apache2`)	1 + n/a	dereference, function, `if`, loop, multiline
7	120348	Gcov	L	1 (`lzma`)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
8	120484	Gcov	L	1 (`curl`)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (`lz4`)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (`lzma`)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (`lzma`)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (`distro-info`)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (`lzma`)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (`lzo2`)	4 + 16	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (`mawk`)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (`mawk`)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (`sl`)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (`grep`)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	L/BV	1 (`hostname`)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (`lzma`)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (`net-tools`)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (`hostname`)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (`lzo2`)	0 + 4	inline function
24	121897	Gcov	L	2 (`ifupdown`)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (`bzip2`)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (`mawk`)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (`ifupdown`)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (`mawk`)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (`distro-info`, `mawk`)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (`lzo2`, `procps`)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (`distro-info`)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (`psmisc`)	0 + 1	loop, multiline

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	9 (`lzo2`)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	2 (`lzo2`)	3 + 2	inline function, multistmt
3	117412	Gcov	L	3 (`lz4`)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (`lz4`)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (`lz4`)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (`apache2`)	1 + n/a	dereference, function, `if`, loop, multiline
7	120348	Gcov	L	1 (`lzma`)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
8	120484	Gcov	L	1 (`curl`)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (`lz4`)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (`lzma`)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (`lzma`)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (`distro-info`)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (`lzma`)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (`lzo2`)	4 + 16	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (`mawk`)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (`mawk`)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (`sl`)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (`grep`)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	L/BV	1 (`hostname`)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (`lzma`)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (`net-tools`)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (`hostname`)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (`lzo2`)	0 + 4	inline function
24	121897	Gcov	L	2 (`ifupdown`)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (`bzip2`)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (`mawk`)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (`ifupdown`)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (`mawk`)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (`distro-info`, `mawk`)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (`lzo2`, `procps`)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (`distro-info`)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (`psmisc`)	0 + 1	loop, multiline

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs
  - In both tools: **20** in Gcov and **12** in LLVM-cov

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	2 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lz4)	3 + 1	dereference, function, multiline, type conversion
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (apache2)	1 + n/a	dereference, function, `if`, loop, multiline
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	2 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lzma)	3 + 1	dereference, function, multiline, type conversion
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (lzma)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lzma)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	2 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	1 (lz4)	3 + 1	dereference, function, multiline, type conversion
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (lz4)	3 + n/a	inline function, `switch-case`
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `recurn`, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (lz4)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lz4)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L/BN	1 (lzma)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	5 (lzo2)	4 + 16	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	3 (mawk)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (sl)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	L/BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	BN	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (hostname)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (lzo2)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bzip2)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (mawk)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple
- Recurring patterns

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	2 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (lz4)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lz4)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L	1 (lz4)	3 + n/a	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	3 (lz4)	4 + n/a	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	1 (lz4)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (lz4)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	L	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (net-tools)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	1 (ifupdown)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bsdmainutils)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (ifupdown)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple
- Recurring patterns
  - Coverage mapping
  - Coding style
  - C++
  - Constants
  - “No code”

Bugs Found by This Work

- **2** crashing bugs in LLVM-cov
- **32** wrong coverage bugs

#	Bug ID	Tool	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	120321	Gcov	L	2 (lzo2)	6 + 11	constant, loop, no code, variable scope
2	120478	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
3	117412	Gcov	L	3 (lz4)	3 + 1	dereference, function, multiline, type conversion
4	120482	Gcov	L	1 (lz4)	3 + n/a	`const` keyword, inline function
5	120490	Gcov	L	1 (lz4)	3 + n/a	inline function, `switch-case`
6	117415	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
7	120348	Gcov	L	1 (lzma)	1 + n/a	C++, `extern "C"`, function, loop, `return`, variable scope
8	120484	Gcov	L	1 (curl)	1 + n/a	`if`, loop, variable scope
9	120489	Gcov	L	1 (lz4)	1 + n/a	`continue`, dereference, loop, variable scope
10	120491	Gcov	L	1 (lz4)	1 + n/a	C++, constructor, multiline
11	120492	Gcov	L	1 (lz4)	1 + n/a	C++ standard library, type conversion
12	120486	Gcov	L/BN	5 (distro-info)	6 + 19	multiline, ternary
13	120319	Gcov	L	1 (lz4)	4 + 0	C++, function, multiline, variable scope
14	120332	Gcov	L/BN/BV	3 (lz4)	4 + 0	`if`, loop, no code, `switch-case`
15	120485	Gcov	BV	1 (mawk)	23 + 0	preprocessor directive, header
16	140427	LLVM-cov	L	1 (lz4)	2 + 2	`goto`, `if`, loop, macro, `return`
17	114622	LLVM-cov	L	1 (lz4)	1 + n/a	`break`, header, macro
18	116884	LLVM-cov	L	1 (grep)	1 + n/a	`break`, constant, loop
19	105341	LLVM-cov	BV	1 (hostname)	1 + n/a	concurrency
20	116902	LLVM-cov	L	1 (lzma)	3 + n/a	C++ standard library, function
21	101241	LLVM-cov	BN/MN	1 (net-tools)	2 + n/a	comma, constant, tautological compare, type conversion
22	131505	LLVM-cov	BN/BV/MN	1 (net-tools)	1 + n/a	dereference, header, macro, type conversion
23	121914	Gcov	L	2 (ifupdown)	0 + 4	inline function
24	121897	Gcov	L	2 (ifupdown)	0 + 3	function, variable scope
25	121901	Gcov	L	1 (bsdtar)	0 + 3	dereference, loop, `malloc`, multiline
26	121896	Gcov	L	1 (ifupdown)	0 + 2	function, `if`, pointer
27	121902	Gcov	L	1 (ifupdown)	0 + 1	`continue`
28	158003	LLVM-cov	L	1 (mawk)	0 + 4	macro, `switch-case`
29	158080	LLVM-cov	L	2 (distro-info, mawk)	0 + 4	`break`, `if`, `switch-case`
30	157959	LLVM-cov	L	2 (lzo2, procps)	0 + 2	`goto`, `if`
31	157946	LLVM-cov	L	1 (distro-info)	0 + 1	preprocessor directive, header, macro
32	157981	LLVM-cov	L	1 (psmisc)	0 + 1	loop, multiline

- In both tools: **20** in Gcov and **12** in LLVM-cov
- Across all 3 metrics; one bug can affect multiple
- Recurring patterns
  - Coverage mapping
  - Coding style
  - C++
  - Constants
  - “No code”

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |     if (g < 256 && h)
    [True: 1, False: 0]
    [True: 0, False: 1]

        C1, C2   Result
    1 { T , F  = F     }

    C1-Pair: not covered
    C2-Pair: not covered

0 |         return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

```
[True: 1, False: 0]
[True: 0, False: 1]
```

```
    C1, C2   Result
1 { T , F  = F     }

C1-Pair: not covered
C2-Pair: not covered
```

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

```
[True: 1, False: 0]
[True: 0, False: 1]
```

```
    C1, C2   Result
1 { T , F  = F     }

C1-Pair: not covered
C2-Pair: not covered
```

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

```
[True: 1, False: 0]
[True: 0, False: 1]
```

```
    C1, C2   Result
1 { T , F  = F     }

C1-Pair: not covered
C2-Pair: not covered
```

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
  1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
  1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

```
test.c:3:11: warning: result of comparison of constant
 256 with expression of type 'unsigned char' is always
 true [-Wtautological-constant-out-of-range-compare]
    3 |      if (g < 256 && h)
      |          ~ ^ ~~~
```

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
  1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
  1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
1 |  int main(void) {
  |      int i = 0;
1 |      while (1) {
6 |          0;
7 |          if (++i >= 7)
1 |              break;
  |
  |      }
  |  }
```

Input program for GCC#120321, reduced from `lzo2` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
  1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
  1 |  int main(void) {
    |      int i = 0;
  1 |      while (1) {
~~6~~ |          0;
  7 |          if (++i >= 7)
  1 |              break;
    |
    |      }
    |  }
```

Input program for GCC#120321, reduced from `lzo2` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
  1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
  1 |  int main(void) {
    |      int i = 0;
  1 |      while (1) {
6 |          0;
  7 |          if (++i >= 7)
  1 |              break;
    |          // implicit "else"
    |      }
    |  }
```

Input program for GCC#120321, reduced from `lzo2` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
 1 { T , F  = F     }
```

~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
1 |  int main(void) {
  |      int i = 0;
1 |      while (1) {
6 |          0;
7 |          if (++i >= 7)
1 |              break;
  |          // implicit "else"
  |      }
  |  }
```

Input program for GCC#120321, reduced from `lzo2` package

Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
```

~~[True: 1, False: 0]~~
[True: 0, False: 1]

```
      C1, C2   Result
 1 { T , F  = F     }
```

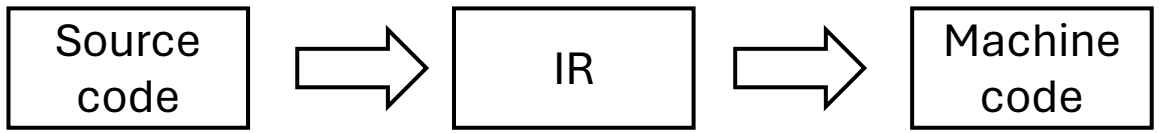
~~C1-Pair: not covered~~
C2-Pair: not covered

```
0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
1 |  int main(void) {
  |      int i = 0;
1 |      while (1) {
6 |          0;
7 |          if (++i >= 7)
1 |              break;
  |          // implicit "else"
  |      }
  |  }
```

Input program for GCC#120321, reduced from `lzo2` package



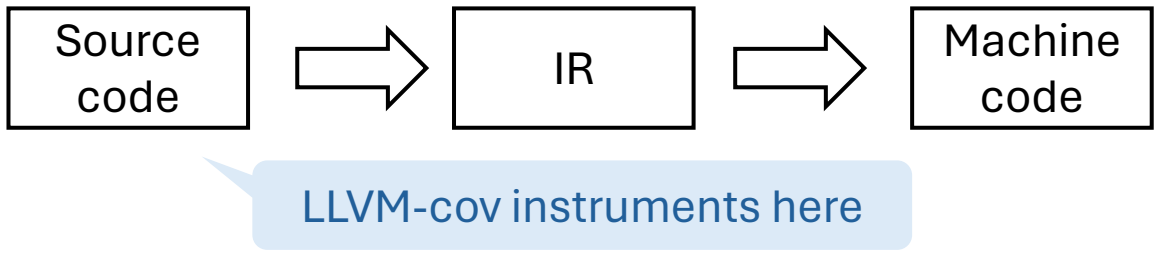
Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
  [True: 1, False: 0]
  [True: 0, False: 1]

      C1, C2   Result
  1 { T , F  = F     }

  C1-Pair: not covered
  C2-Pair: not covered
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

```
1 |  int main(void) {
  |      int i = 0;
1 |      while (1) {
6 |          0;
7 |          if (++i >= 7)
1 |              break;
  |          // implicit "else"
  |      }
  |  }
```

Input program for GCC#120321,
reduced from `lzo2` package



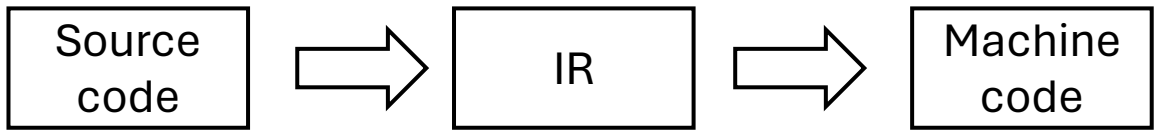
Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
  [True: 1, False: 0]
  [True: 0, False: 1]

      C1, C2   Result
  1 { T , F  = F     }

  C1-Pair: not covered
  C2-Pair: not covered
0 |          return 1;
1 |  }
```

Input program for LLVM#101241,
reduced from `net-tools` package

```
1 |  int main(void) {
  |      int i = 0;
1 |      while (1) {
6 |          0;
7 |          if (++i >= 7)
1 |              break;
  |          // implicit "else"
  |      }
  |  }
```

Input program for GCC#120321,
reduced from `lzo2` package



LLVM-cov instruments here

No sophisticated analysis

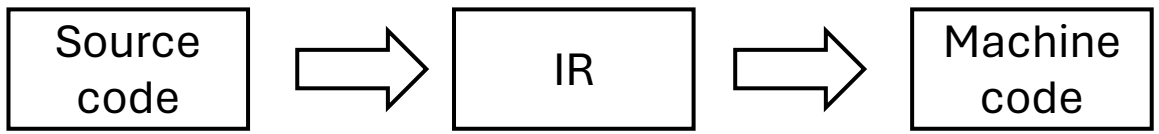
Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
  [True: 1, False: 0]
  [True: 0, False: 1]

      C1, C2   Result
  1 { T , F  = F     }

  C1-Pair: not covered
  C2-Pair: not covered

0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
  1 |  int main(void) {
    |      int i = 0;
  1 |      while (1) {
  6 |          0;
  7 |          if (++i >= 7)
  1 |              break;
    |          // implicit "else"
    |      }
    |  }
```

Input program for GCC#120321, reduced from `lzo2` package



Gcov instruments here

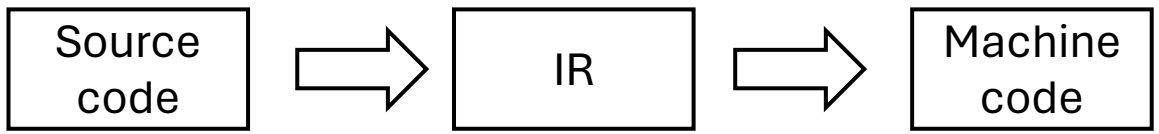
Example Bugs

```
  |  unsigned char g, h;
1 |  int main(void) {
1 |      if (g < 256 && h)
  [True: 1, False: 0]
  [True: 0, False: 1]

      C1, C2   Result
  1 { T , F  = F     }

  C1-Pair: not covered
  C2-Pair: not covered

0 |          return 1;
1 |  }
```

Input program for LLVM#101241, reduced from `net-tools` package

```
  1 |  int main(void) {
    |      int i = 0;
  1 |      while (1) {
  6 |          0;
  7 |          if (++i >= 7)
  1 |              break;
    |          // implicit "else"
    |      }
    |  }
```

Input program for GCC#120321, reduced from `lzo2` package



Gcov instruments here

Confused by source code that doesn't end up in the binary

Convention Differences

- Benign differences where both tools make their sense

#	Short Description	Incons. Type	Affected Package(s)	Occur.	Triggering Condition(s)
1	Complex control flow in macros	L	4 (`inetutils`, `lzma`)	9	`goto`, `if`, loop, macro
2	`do-while` statements	L	5 (`hostname`)	6	`if`, loop
3	`else if`	L	2 (`gzip`)	3	`if`
4	Multiline statements	L/BN/BV	19 (`iproute2`)	283	multiline, ternary
5	`switch-case` statements	L/BN/BV	12 (`mawk`)	142	no code, `switch-case`
6	Macros in decisions	BN	11 (`apache2`)	150	macro
7	Decisions in macros	BN	12 (`lz4`)	141	macro
8	Branches in inline functions	BN	1 (`lz4`)	118	inline function
9	Branches for assertions	BN	6 (`lz4`)	52	assertion
10	System header files	BN	7 (`iproute2`)	44	header
11	Constant condition	BN/MN	11 (`iproute2`)	58	constant
12	MC/DC ternary	MN	8 (`inetutils`)	12	ternary
13	MC/DC empty “then” clause	MN	3 (`gzip`)	6	`if`, no code
14	MC/DC inline function	MN	1 (`lz4`)	1	`const` keyword, inline function
15	Masking vs. unique-cause MC/DC	MV	1 (`distro-info`)	1	mixed conjunctive and disjunctive

Convention Differences

- Benign differences where both tools make their sense

Convention Differences

- Benign differences where both tools make their sense

  - **Example 1:** line coverage for `switch`-`case` statements

Convention Differences

- Benign differences where both tools make their sense

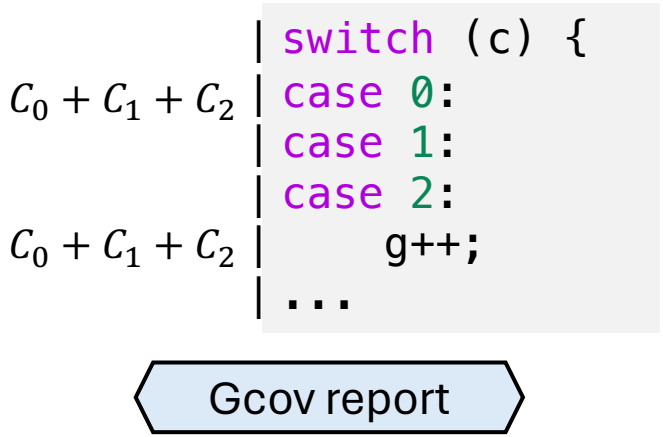
- **Example 1:** line coverage for `switch-case` statements

```
switch (c) {
case 0:
case 1:
case 2:
    g++;
...
```

Convention Differences

- Benign differences where both tools make their sense

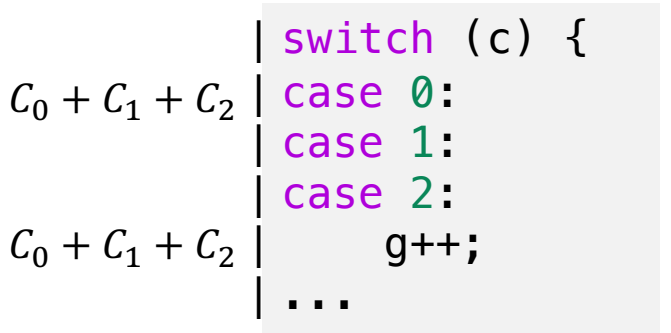
- **Example 1:** line coverage for `switch-case` statements



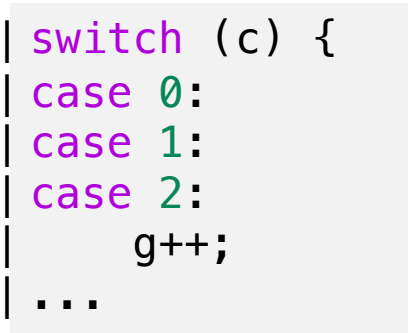
Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements



Gcov report

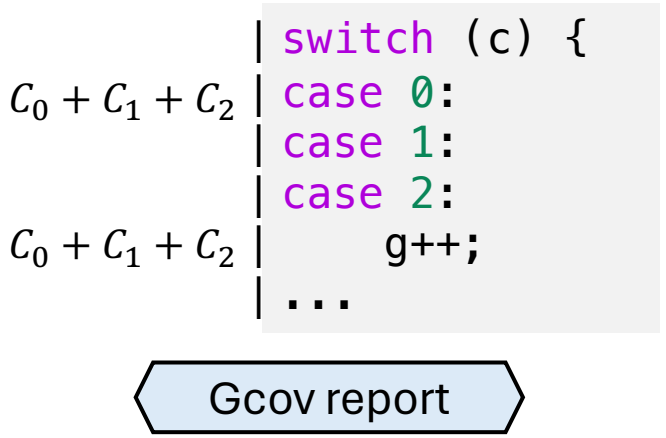


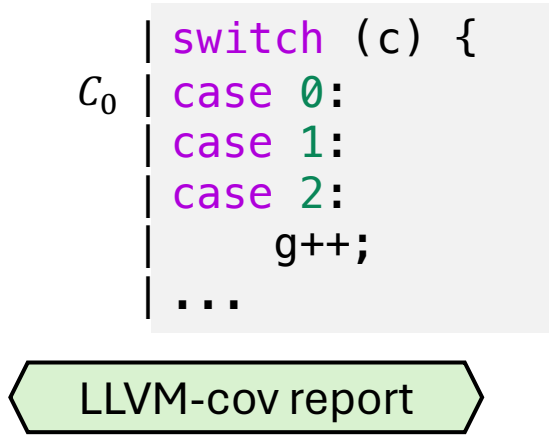
LLVM-cov report

Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements

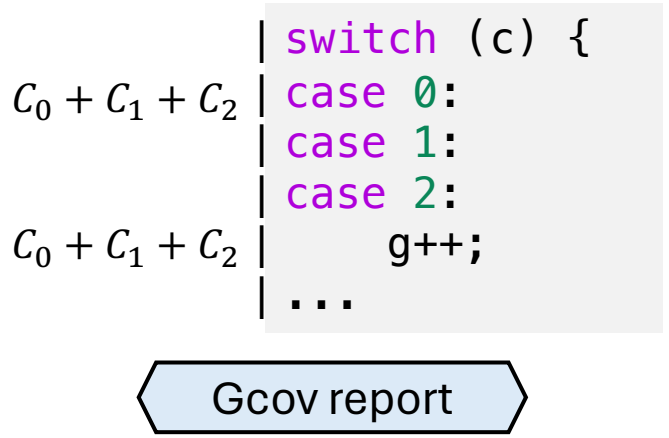


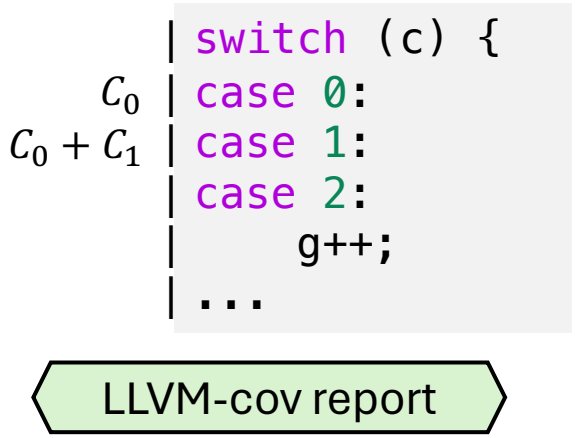


Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements

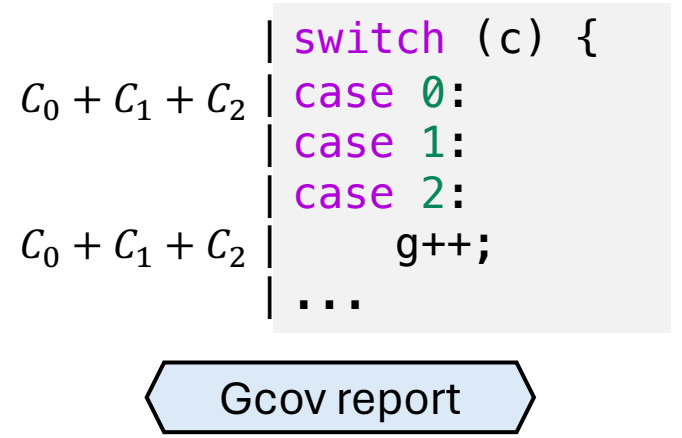


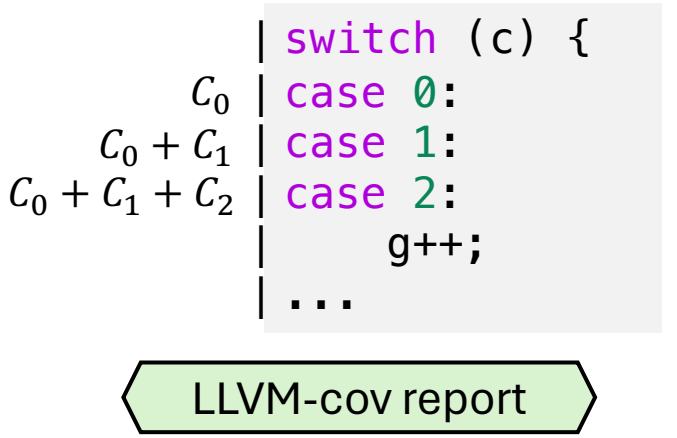


Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements

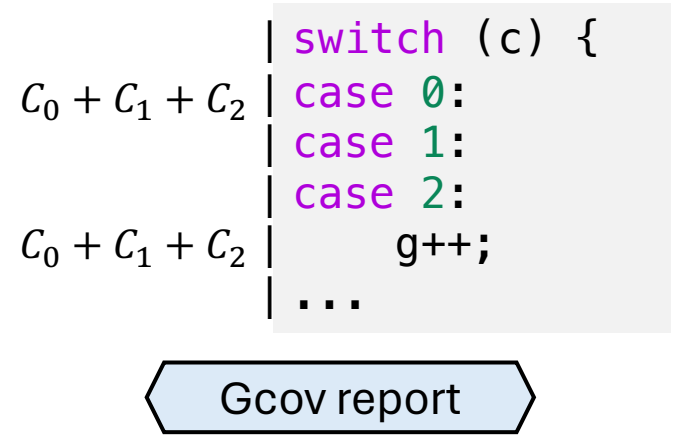


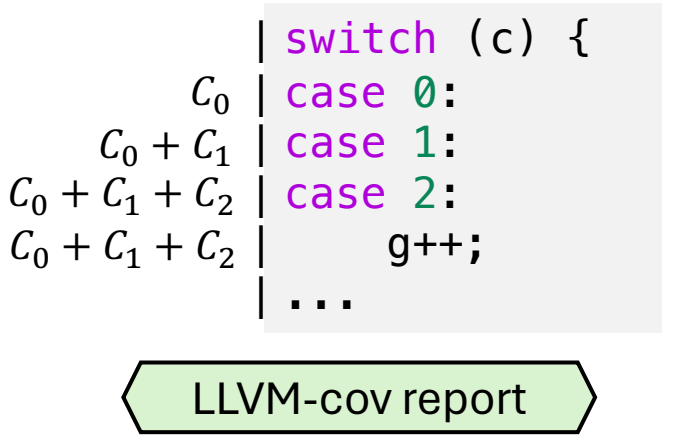


Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements

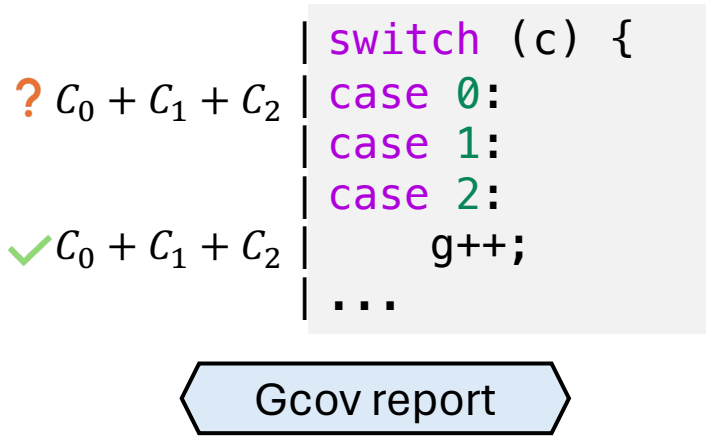


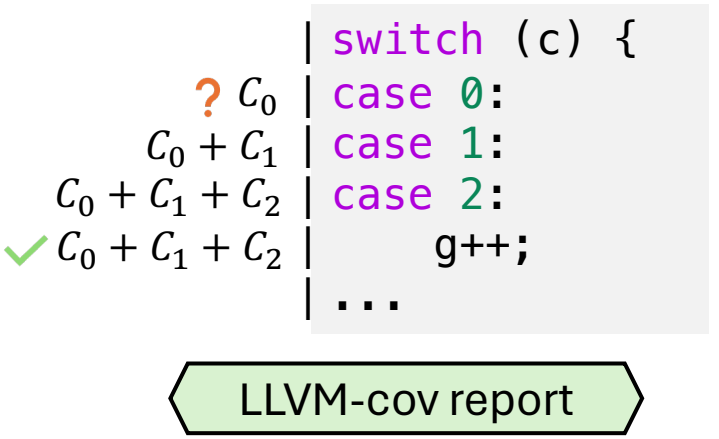


Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements

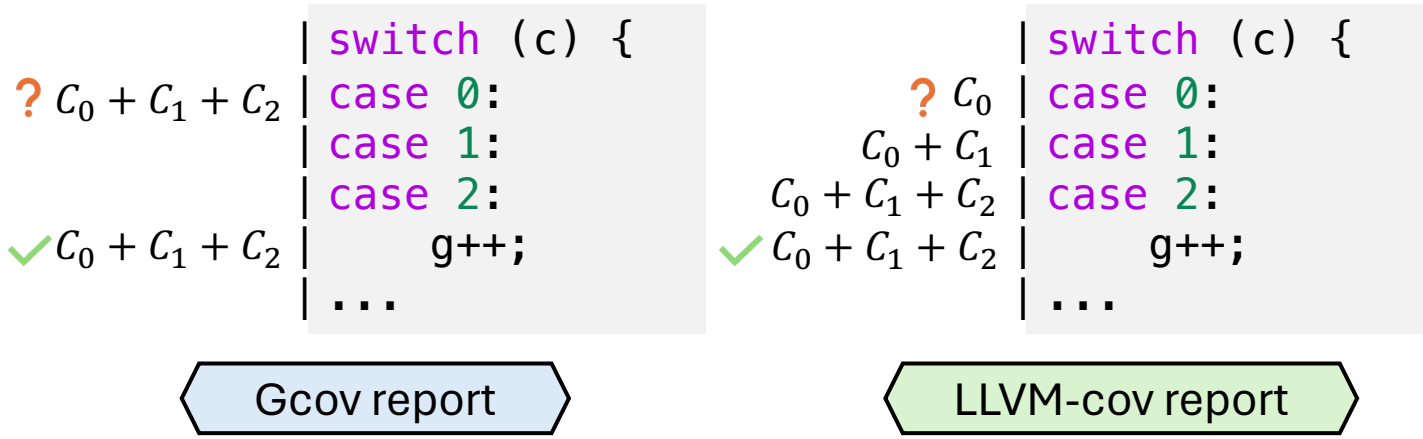




Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements

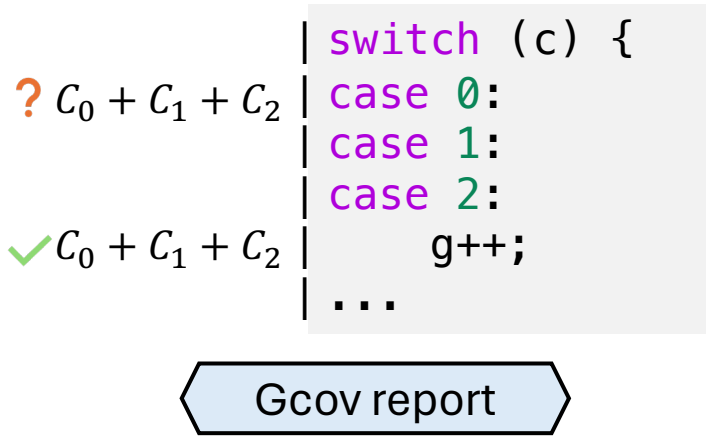


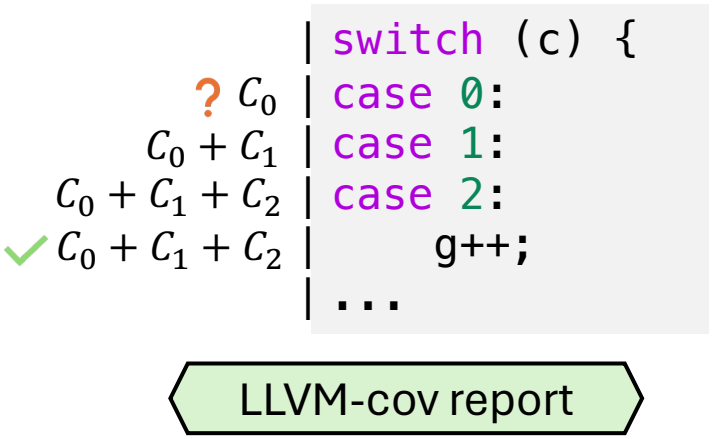
- **Example 2:** LLVM-cov can under-report MC/DC for certain expressions

Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch-case` statements





- **Example 2:** LLVM-cov can under-report MC/DC for certain expressions

It turns out Gcov and LLVM-cov adopt **two variants** of MC/DC definition, i.e., masking MC/DC and unique-cause MC/DC [Chilenski, 2001]

Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch`-`case` statements

```
switch (c) {
case 0:
```

? $C_0 + C_1 + C_2$

? C_0

✓ $C_0 + C$

Developer clarification
https://discourse.llvm.org/t/rfc-source-based-mc-dc-code-coverage/59244/15

evodius96 1 whentojump Jun 2024

Hi @whentojump !

> - Q: Can you please confirm LLVM is currently following the unique-cause criteria?

I believe this is correct, as far as I understand; what I implemented in clang follows what other vendors have supported, especially LDRA. There aren't any improvements to mask out conditions using boolean logic evaluation of whole subexpressions. However, in the design doc, I inaccurately described it as *masking*, but this only applies to the fact that due to the short-circuit behavior of logical operators, unevaluatable conditions are masked-out and counted as dont-cares.

- **Example 2:** LLVM-cov can under-report MC/DC for certain expressions

Convention Differences

- Benign differences where both tools make their sense

- **Example 1:** line coverage for `switch`-`case` statements

? $C_0 + C_1 + C_2$

```
switch (c) {
case 0:
```

✓ $C_0 + C$

? C_0

```
switch (c) {
case 0:
```

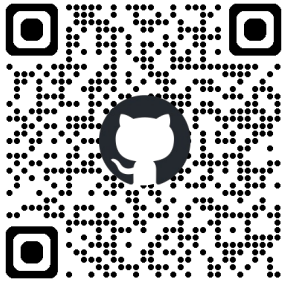
Developer clarification
https://discourse.llvm.org/t/rfc-source-based-mc-dc-code-coverage/59244/15

cares.

A standard on code coverage (just like C/C++ standard) is much desired

Summary

- Real-world projects help reveal unique coverage tool bugs
- DEBCOVDIFF: a differential testing framework
- Revealed 34 new bugs in Gcov and LLVM-cov
- Open sourced at https://github.com/xlab-uiuc/DEBCOVDIFF



Future Directions

- More challenging input than Debian packages, such as Linux kernel, Xen hypervisor
- Understand input dynamics using the calibrated coverage